```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.ExileFarm.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S26_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
            // T010 wiring: the procedural pages host owns the four frozen UI pages.
            PagesHost.EnsureInScene();
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.ExileFarm.Boot
```

```
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
```

```
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.ExileFarm.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
```

```
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
```

```
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// AdCore.cs
// AdCore.cs - S1 T009 ad-slot policy core for ExileFarm (B1277).
// Ported under the kit-port law: the E4 kit carries no ad-slot code (the kit ad
// face is the ads=0 closure anchor - zero forced ads, the loop closes clean);
// every frozen value here traces to S07 / U149 / the release-plan revenue row:
//   - four slot ids = requirement S07 verbatim: AD_SPEED_UP / AD_RESTOCK /
//     AD_DOUBLE / AD_OFFLINE_EXPAND (release-plan four rewarded slots: growth
//     speed-up / restock-reseed / income double / offline-summary expand)
//   - DailyCap 12 = U149 verbatim (12-per-day hard top, the domestic rule)
//   - WindowSeconds 86400 = S07 rolling wall-clock window verbatim
//   - cap-reject = silent zero mutation (U149: the capped player simply sees no
//     new offer - no event, no counter, no ledger touch, capsule untouched)
//   - zero system blocking = structural (S07 horizontal system): AdCore holds no
//     reference to any other core and mutates nothing but its own ledger,
//     capsules and events; EpisodeCore keeps its own offer events (T005 seam
//     note: the AdCapsule policy is the T009 seam and stays out).
// Save face: save.adcaps = the grant ledger only (wall-clock long seconds);
// LoadLedger prunes at load (the across-sessions face - stale grants die at
// session start, the cap never inherits a dead window).
// Pure C#: zero UnityEngine; the wall clock is injected (double seconds), which
// keeps the whole policy offline and testable (red line 12 - zero server).
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class AdSlotEvent
    {
        public double t;
        public string type;
        public string detail;
        public AdSlotEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    // an open ad offer: one capsule per offer, closed exactly once (declined or
    // granted) - a terminal capsule never re-opens, a decline never blocks
    public sealed class AdCapsule
    {
        public string Slot;
        public double CreatedTs;
        public int State; // 0 open, 1 declined, 2 granted
        public AdCapsule(string slot, double ts) { Slot = slot; CreatedTs = ts; State = 0; }
    }
    public sealed class AdCore
    {
        // ---- frozen anchors (S07 / U149 verbatim) ----
```

```
public const int DailyCap = 12;                 // U149 12-per-day hard top
public const double WindowSeconds = 86400.0;     // S07 rolling wall-clock window
public static readonly string[] FrozenSlots =
{
    "AD_SPEED_UP", "AD_RESTOCK", "AD_DOUBLE", "AD_OFFLINE_EXPAND"
};
public static readonly string[] SlotFaces =
{
    "growth speed-up", "restock and reseed", "income double", "offline summary expand"
};
public double WallClock;   // injected wall-clock seconds (the rolling-window domain)
public readonly List&lt;AdSlotEvent&gt; Events = new List&lt;AdSlotEvent&gt;();
public readonly List&lt;long&gt; GrantTs = new List&lt;long&gt;();   // save.adcaps face
public int OffersMade, OffersDeclined, GrantsTotal, RejectsTotal, PrunedCount;
public void Log(string type, string detail) { Events.Add(new AdSlotEvent(WallClock, type, detail)); }
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
public static int SlotIndex(string slot)
{
    if (slot == null) return -1;
    for (int i = 0; i &lt; FrozenSlots.Length; i++) if (FrozenSlots[i] == slot) return i;
    return -1;
}
// grants inside the rolling window: ts in (WallClock - Window, WallClock]
public int GrantsInWindow()
{
    double edge = WallClock - WindowSeconds;
    int n = 0;
    for (int i = 0; i &lt; GrantTs.Count; i++)
        if ((double)GrantTs[i] &gt; edge &amp;&amp; (double)GrantTs[i] &lt;= WallClock) n++;
    return n;
}
// silent plumbing: stale grants (ts &lt;= edge) fall out of the ledger
public void Prune()
{
    double edge = WallClock - WindowSeconds;
    for (int i = GrantTs.Count - 1; i &gt;= 0; i--)
    {
        if ((double)GrantTs[i] &lt;= edge)
        {
            GrantTs.RemoveAt(i);
            PrunedCount++;
        }
    }
}
// save face: load the persisted ledger, then prune at load (across sessions)
```

```
public void LoadLedger(long[] saved)
{
    GrantTs.Clear();
    if (saved != null)
        for (int i = 0; i &lt; saved.Length; i++) GrantTs.Add(saved[i]);
    Prune();
}
public long[] ExportLedger() { return GrantTs.ToArray(); }
// ==== offer face: unknown slot = one loud reject; at cap = SILENT zero
// mutation (U149 cap-reject law - the capped player sees no offer at all)
public AdCapsule OfferAd(string slot)
{
    int idx = SlotIndex(slot);
    if (idx &lt; 0)
    {
        RejectsTotal++;
        Log("ad_rejected", "unknown slot '" + slot + "'");
        return null;
    }
    if (GrantsInWindow() &gt;= DailyCap) return null;   // silent: no event, no counter, no ledger touch
    OffersMade++;
    AdCapsule c = new AdCapsule(slot, WallClock);
    Log("ad_offered", "slot " + slot + " offered: " + SlotFaces[idx] + " - rewarded, purely optional, zero block
    return c;
}
// ==== grant face: the hard top binds here too - an accept at cap is a
// silent zero-mutation false (capsule stays open, nothing moves)
public bool AcceptOffer(AdCapsule c)
{
    if (c == null) { RejectsTotal++; Log("ad_rejected", "null capsule"); return false; }
    if (c.State != 0) { RejectsTotal++; Log("ad_rejected", "capsule already closed (state " + c.State + ")"); re
    int idx = SlotIndex(c.Slot);
    if (idx &lt; 0) { RejectsTotal++; Log("ad_rejected", "capsule carries unknown slot '" + c.Slot + "'"); retur
    if (GrantsInWindow() &gt;= DailyCap) return false;   // silent hard top
    GrantTs.Add((long)WallClock);
    c.State = 2;
    GrantsTotal++;
    Log("ad_granted", "rewarded grant slot=" + c.Slot + " face=" + SlotFaces[idx]
        + " - window count " + GrantsInWindow() + "/" + DailyCap);
    return true;
}
// ==== decline face: loud player action, full closure (a declined capsule
// never grants; a fresh offer is always possible - purely optional)
public bool DeclineOffer(AdCapsule c)
{
    if (c == null) { RejectsTotal++; Log("ad_rejected", "null capsule"); return false; }
    if (c.State != 0) { RejectsTotal++; Log("ad_rejected", "capsule already closed (state " + c.State + ")"); re
    c.State = 1;
    OffersDeclined++;
    Log("ad_declined", "slot " + c.Slot + " declined - purely optional, the game never asks twice in a row");
```

```
            return true;
        }
    }
}
// AudioHooks.cs
// AudioHooks.cs - S1 T010 c2 audio hook true-path core for ExileFarm (B1284).
// The audio work orders (demand-doc section 5, 11 stems) are TJ-custom assets that
// have not landed yet - this core IS the frozen contract they will plug into:
// the frozen stem census (11 manifest ids verbatim) plus the event-&gt;stem map that
// turns the REAL core event streams into audio hook records. True path = the hook
// consumer walks the same event lists the checks and the presenter walk - zero
// mocked sources, zero UnityEngine (no AudioClip face exists yet by design: the
// platform layer will later subscribe to these hook records and play the stem
// assets by name when they arrive).
// Mapping law (section 5, zero re-tuning): the field run start starts the
// frontier BGM; the kit sfx faces map onto the till / plant feedback stems; the
// 1.50 same-beat triple member grain_pop and the harvest taps both ring the
// harvest bell (AU-04 is a member of the triple and the harvest face); the
// estate upgrade, episode unlock and cliffhanger beats map one-to-one; the loss
// family (near_miss / episode_lost_clean / run_lost) shares the warm loss stem
// (AU-08 warm-not-scary by design); every NEW dex entry fires the dex-unlock
// chime; the offline summary rings the letter chime; the UI tap face is an
// explicit call the page button layer will wire. Everything else stays silent
// by law: reject events are silent (zero dead-tap face), ad offer / accept /
// decline events are silent (U149 - the loop never requires audio ads), goal
// and credit events carry no stem in the 11-order census.
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class AudioHook
    {
        public double t;
        public string stem;
        public string source;
        public AudioHook(double t, string stem, string source) { this.t = t; this.stem = stem; this.source = source; }
    }
    public sealed class AudioHooks
    {
        // ---- frozen stem census (demand-doc section 5, manifest ids verbatim) ----
        public const int StemCount = 11;
        public static readonly string[] Stems =
        {
            "g26_bgm_frontier", "g26_till", "g26_plant", "g26_harvest", "g26_upgrade",
            "g26_episode_unlock", "g26_cliffhanger", "g26_near_miss", "g26_dex_unlock",
            "g26_ui_tap", "g26_offline_chime"
        };
        public readonly List&lt;AudioHook&gt; Fired = new List&lt;AudioHook&gt;();
        public AudioHooks()
        {
```

```
    // frozen census guard: the 11 stems must stay unique and non-empty
    for (int i = 0; i &lt; Stems.Length; i++)
    {
        if (Stems[i] == null || Stems[i].Length == 0)
            throw new ArgumentException("audio stem " + i + " must carry the manifest id");
        for (int j = i + 1; j &lt; Stems.Length; j++)
            if (Stems[i] == Stems[j])
                throw new ArgumentException("audio stem census must stay unique: " + Stems[i]);
    }
}
// ---- per-core allowed maps (frozen; a stem only fires from its real
// source stream - run_start maps ONLY from the field face so one run
// starts the BGM exactly once even though the episode and dex cores log
// their own run_start rows) ------------------------------------------
static readonly string[] FieldMapTypes = { "run_start", "sfx_hoe", "sfx_seed", "grain_pop", "harvest_tap" };
static readonly string[] FieldMapStems = { "g26_bgm_frontier", "g26_till", "g26_plant", "g26_harvest", "g26_harv
static readonly string[] EpisodeMapTypes = { "episode_unlock", "cliffhanger_fire", "near_miss", "episode_lost_cl
static readonly string[] EpisodeMapStems = { "g26_episode_unlock", "g26_cliffhanger", "g26_near_miss", "g26_near
static readonly string[] EstateMapTypes = { "estate_upgrade" };
static readonly string[] EstateMapStems = { "g26_upgrade" };
static readonly string[] CodexMapTypes = { "dex_unlock", "offline_summary", "run_lost" };
static readonly string[] CodexMapStems = { "g26_dex_unlock", "g26_offline_chime", "g26_near_miss" };
static string StemOf(string[] types, string[] stems, string type)
{
    for (int i = 0; i &lt; types.Length; i++) if (types[i] == type) return stems[i];
    return null;   // unmapped = silent by law
}
/// &lt;summary&gt;Consume the field event stream (real source, same order, same
/// ticks). The BGM starts with the run; the kit sfx faces ring the till /
/// plant stems; the 1.50 triple member grain_pop and every harvest tap ring
/// the harvest bell.&lt;/summary&gt;
public void ConsumeField(FieldCore core)
{
    if (core == null) return;
    for (int i = 0; i &lt; core.Events.Count; i++)
    {
        string stem = StemOf(FieldMapTypes, FieldMapStems, core.Events[i].type);
        if (stem != null) Fired.Add(new AudioHook(core.Events[i].t, stem, "field:" + core.Events[i].type));
    }
}
/// &lt;summary&gt;Consume the episode event stream: unlock / cliffhanger / the
/// loss family. Ad offer + rescue + double offers stay silent (U149).&lt;/summary&gt;
public void ConsumeEpisode(EpisodeCore core)
{
    if (core == null) return;
    for (int i = 0; i &lt; core.Events.Count; i++)
    {
        string stem = StemOf(EpisodeMapTypes, EpisodeMapStems, core.Events[i].type);
        if (stem != null) Fired.Add(new AudioHook(core.Events[i].t, stem, "episode:" + core.Events[i].type));
    }
```

```
        }
        /// &lt;summary&gt;Consume the estate event stream: the five-stage upgrade ring.&lt;/summary&gt;
        public void ConsumeEstate(EstateCore core)
        {
            if (core == null) return;
            for (int i = 0; i &lt; core.Events.Count; i++)
            {
                string stem = StemOf(EstateMapTypes, EstateMapStems, core.Events[i].type);
                if (stem != null) Fired.Add(new AudioHook(core.Events[i].t, stem, "estate:" + core.Events[i].type));
            }
        }
        /// &lt;summary&gt;Consume the dex event stream: every NEW dex entry rings the
        /// unlock chime (the survey of a fresh run fires twelve, a re-survey fires
        /// zero - idempotence is the audio face of the zero-punishment wall); the
        /// offline summary rings the letter chime; run_lost joins the loss family.&lt;/summary&gt;
        public void ConsumeCodex(CodexCore core)
        {
            if (core == null) return;
            for (int i = 0; i &lt; core.Events.Count; i++)
            {
                string stem = StemOf(CodexMapTypes, CodexMapStems, core.Events[i].type);
                if (stem != null) Fired.Add(new AudioHook(core.Events[i].t, stem, "codex:" + core.Events[i].type));
            }
        }
        /// &lt;summary&gt;UI tap face: the page button layer calls this on every tap
        /// (wall-clock domain, t = -1 - it is not a sim beat). Frozen stem only.&lt;/summary&gt;
        public void UiTap()
        {
            Fired.Add(new AudioHook(-1.0, "g26_ui_tap", "ui:tap"));
        }
        public int CountStem(string stem)
        {
            int n = 0;
            for (int i = 0; i &lt; Fired.Count; i++) if (Fired[i].stem == stem) n++;
            return n;
        }
        public bool HookExistsAt(string stem, double t)
        {
            for (int i = 0; i &lt; Fired.Count; i++)
                if (Fired[i].stem == stem &amp;&amp; Math.Abs(Fired[i].t - t) &lt; 1e-9) return true;
            return false;
        }
    }
}
// CodexCore.cs
// CodexCore.cs - S1 T007 land/crop dex keep core for ExileFarm (B1274 kit-port).
// Ported under the kit-port law: every frozen value stays verbatim - zero new tuning.
// Frozen anchors (all traceable to the E4 kit, Design/evidence/m6_exilefarm_graybox_kit/):
//   - land dex 12 / crop dex 3 = kit PlotTerrainNames (12 rows) + CropCount (3),
//     "never wiped on loss" (kit field comments verbatim): a lost run never wipes
```

```
//     the dex - the zero-punishment meta face (requirement M07 "after a loss the
//     12-slot codex keeps everything with zero punishment", S04 CodexState
//     AddOwned / QueryAll faces).
//   - run survey = kit StartRun face verbatim: every run surveys the twelve border
//     plots and adds each terrain to the dex (per-run terrain permutation
//     ((2*runIdx+i)%12+12)%12 lives in FieldCore - the dex only observes it).
//   - crop observe = kit PlantPlot face verbatim: CropDex.Add(cropIdx % 3).
//   - run lost face = kit RunLost verbatim: "run N ends, the land dex keeps
//     everything" - the loss is an emotion beat, never a punishment (requirement
//     M07 "loss events are an emotion mechanism, not a punishment mechanism").
//   - run won face = kit RunWon verbatim: "five episodes cleared - the wasteland
//     is an estate now".
//   - punish token table = kit PunishTokens verbatim: the whole event stream must
//     stay clean of punish / penalty / confiscate / land_reset / dex_wipe /
//     streak_break (E4 assertion 10 face).
//   - one load = kit BaseYield (100 shi): the offline summary derives its loads
//     from the grain ledger delta / 100 - the frozen copy face "three loads"
//     (i18n offline_summary, "while you were away the frontier brought in three
//     loads of grain").
// Structural S1 extension vs the kit: the offline summary hook face (S05
// return-visit hook) - the kit carried the line only in the i18n table; the
// production core derives loads from the real ledger and fires one
// offline_summary event. Zero threshold change.
// State consumption: the dex observes FieldCore (terrain survey / crop ids) and
// never mutates it; the loss faces are called by the presenter at run end. The
// dex survives StartRun (per-run re-arm resets nothing here - the dex is the
// cross-run meta face). Pure C#: zero UnityEngine.
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class CodexEvent
    {
        public double t;
        public string type;
        public string detail;
        public CodexEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public sealed class CodexCore
    {
        // ---- frozen counts (kit PlotTerrainNames / CropCount mirror) ----
        public const int FrozenTerrainCount = 12;  // land dex: twelve terrains, full wall
        public const int FrozenCropCount = 3;      // crop dex: millet / bean / wheat
        public const long LoadShi = 100;           // one load of grain = kit BaseYield shi
        // ---- frozen punish token table (kit PunishTokens verbatim) ----
        public static readonly string[] PunishTokens =
        {
            "punish", "penalty", "confiscate", "land_reset", "dex_wipe", "streak_break"
        };
        public double SimTime;
```

```
public readonly List&lt;CodexEvent&gt; Events = new List&lt;CodexEvent&gt;();
public readonly HashSet&lt;int&gt; TerrainDex = new HashSet&lt;int&gt;();   // land dex, never wiped on loss
public readonly HashSet&lt;int&gt; CropDex = new HashSet&lt;int&gt;();     // crop dex, never wiped on loss
public int RunCount, RunWins, RunLosses;
public CodexCore(string[] terrainRoster, int cropCount)
{
    if (terrainRoster == null)
        throw new ArgumentException("terrain roster must carry exactly " + FrozenTerrainCount + " rows");
    if (terrainRoster.Length != FrozenTerrainCount)
        throw new ArgumentException("terrain roster must carry exactly " + FrozenTerrainCount
            + " rows, got " + terrainRoster.Length);
    if (cropCount != FrozenCropCount)
        throw new ArgumentException("crop count must be exactly " + FrozenCropCount
            + ", got " + cropCount);
}
public void Log(string type, string detail) { Events.Add(new CodexEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
// the zero-punish scan face (kit ContainsAny(PunishTokens) verbatim)
public static bool ContainsPunishToken(string text)
{
    if (text == null) return false;
    for (int i = 0; i &lt; PunishTokens.Length; i++) if (text.Contains(PunishTokens[i])) return true;
    return false;
}
// ==== S04 AddOwned faces (idempotent: returns true only on a new entry) ====
// T010 c2 additive face: every NEW dex entry logs a dex_unlock event - the
// audio hook true path for the AU-09 dex-unlock chime. Idempotent re-adds
// stay silent (the wall never re-chimes a survey it already owns).
public bool AddTerrainOwned(int terrainIdx)
{
    if (TerrainDex.Add(terrainIdx))
    {
        Log("dex_unlock", "land dex entry " + FieldCore.PlotTerrainNames[terrainIdx] + " joins the wall");
        return true;
```

```
    }
    return false;
}
public bool AddCropOwned(int cropIdx)
{
    if (CropDex.Add(cropIdx))
    {
        Log("dex_unlock", "crop dex entry " + FieldCore.CropNames[cropIdx] + " joins the wall");
        return true;
    }
    return false;
}
// ==== S04 QueryAll faces ====
public bool HasTerrain(int terrainIdx) { return TerrainDex.Contains(terrainIdx); }
public bool HasCrop(int cropIdx) { return CropDex.Contains(cropIdx); }
public int[] QueryTerrainAll()
{
    var list = new List&lt;int&gt;(TerrainDex);
    list.Sort();
    return list.ToArray();
}
public int[] QueryCropAll()
{
    var list = new List&lt;int&gt;(CropDex);
    list.Sort();
    return list.ToArray();
}
// ==== run survey = kit StartRun survey face verbatim: the presenter calls
// this right after FieldCore.StartRun - the twelve plots of border land are
// surveyed and every terrain joins the land dex. The dex is the cross-run
// meta face: the survey is idempotent and a loss never wipes it.
public void SurveyRun(FieldCore field, int runIdx)
{
    if (field == null) return;
    RunCount++;
    for (int i = 0; i &lt; FieldCore.PlotCount; i++)
        if (TerrainDex.Add(field.PlotTerrain[i]))
            Log("dex_unlock", "land dex entry " + FieldCore.PlotTerrainNames[field.PlotTerrain[i]] + " joins the
    Log("run_start", "run " + runIdx + ": exiled to the frontier - twelve plots of border land are surveyed");
}
// ==== crop observe = kit PlantPlot CropDex face verbatim: every plant adds
// its crop to the crop dex (crop rotation keeps all three alive). The
// T010 c2 additive face: a first-time DIRECT roster index rings a
// dex_unlock event; a folded input (out-of-roster, cropIdx % CropCount)
// joins the dex silently - the fold face never rings (T010 c2 A3 frozen
// acceptance: re-survey + folds + idempotent re-adds fire zero).
public void ObservePlant(int cropIdx)
{
    int crop = ((cropIdx % FrozenCropCount) + FrozenCropCount) % FrozenCropCount;
    if (cropIdx &lt; 0 || cropIdx &gt;= FrozenCropCount)
```

```
    {
        CropDex.Add(crop);   // folded face: joins the wall, never rings
        return;
    }
    if (CropDex.Add(crop))
        Log("dex_unlock", "crop dex entry " + FieldCore.CropNames[crop] + " joins the wall");
}
// ==== kit RunLost verbatim: the run ends, the dex keeps everything. The
// loss never touches TerrainDex / CropDex / the grain ledger - zero
// punishment is structural (there is no wipe call anywhere).
public void RunLost(int runIdx)
{
    RunLosses++;
    Log("run_lost", "run " + runIdx + " ends, the land dex keeps everything");
}
// ==== kit RunWon verbatim ====
public void RunWon()
{
    RunWins++;
    Log("run_won", "five episodes cleared - the wasteland is an estate now");
}
// ==== S05 offline summary hook (structural S1 extension): the return-visit
// face. Loads derive from the real grain ledger delta / LoadShi (kit
// BaseYield) - the frozen copy face is three loads (300 shi while away).
// Fires exactly one event when the delta carries at least one load;
// a short delta stays silent (zero spam).
public bool OfflineSummary(long grainAtLastSeen, long grainNow)
{
    long delta = grainNow - grainAtLastSeen;
    if (delta &amp;lt; LoadShi) return false;
    long loads = delta / LoadShi;
    Log("offline_summary", "while you were away the frontier brought in " + loads
        + (loads == 1 ? " load" : " loads") + " of grain (" + delta + " shi)");
    return true;
}
// ==== S2 T014 codex wall (twelve-grid content face, additive - zero S1
// drift): crop family 6 + item family 4 + people family 2 (requirement
// S2 codex table). The people rows carry only the two fictional frontier
// roles (village chief / caravan leader) - zero real dynasty figures,
// guardrail #2. One wall row unlocks per episode clear (unlock_cond
// "episode_clear:N", one row per episode): the presenter calls
// OnEpisodeCleared right after an episode resolves alive. The wall is
// the cross-run meta face like the land/crop dex - never wiped on loss
// (zero punishment is structural: there is no wipe call anywhere).
public const int FrozenWallCount = 12;
public readonly HashSet&amp;lt;int&amp;gt; WallDex = new HashSet&amp;lt;int&amp;gt;();
private string[] _wallNames = new string[0];
private int[] _wallEpisodes = new int[0];
public void InitWall(string[] wallNames, int[] unlockEpisodes)
{
```

```
            if (wallNames == null || unlockEpisodes == null)
                throw new ArgumentException("codex wall roster must carry exactly " + FrozenWallCount + " rows");
            if (wallNames.Length != FrozenWallCount || unlockEpisodes.Length != FrozenWallCount)
                throw new ArgumentException("codex wall roster must carry exactly " + FrozenWallCount
                    + " rows, got " + wallNames.Length + "/" + unlockEpisodes.Length);
            _wallNames = wallNames;
            _wallEpisodes = unlockEpisodes;
        }
        public bool HasWall(int rowIdx) { return WallDex.Contains(rowIdx); }
        public int WallCount { get { return WallDex.Count; } }
        public int[] QueryWallAll()
        {
            var list = new List&lt;int&gt;(WallDex);
            list.Sort();
            return list.ToArray();
        }
        // presenter face: the first clear of an episode joins its wall row and
        // rings one dex_unlock event (the AU-09 chime path, same event family as
        // the land/crop dex); re-clears stay silent (the wall never re-chimes).
        public bool OnEpisodeCleared(int episodeIdx)
        {
            for (int i = 0; i &lt; _wallEpisodes.Length; i++)
            {
                if (_wallEpisodes[i] != episodeIdx) continue;
                if (WallDex.Add(i))
                {
                    Log("dex_unlock", "codex wall entry " + _wallNames[i] + " joins the wall");
                    return true;
                }
                return false;
            }
            return false;
        }
    }
}
// EpisodeCore.cs
// EpisodeCore.cs - S1 T005 season-episode serialization core for ExileFarm (B1271 kit-port).
// Ported under the kit-port law: every frozen value stays verbatim - zero new tuning.
// Frozen anchors (all traceable to the E4 kit, Design/evidence/m6_exilefarm_graybox_kit/):
//   - timeline teach beats (M05 / the six-beat second-by-second table): EpisodeUnlockT
//     5.50s = episode 1 unlock (the serialization teach beat) and CliffhangerT 8.00s =
//     episode 1 complete -&gt; next-episode teaser fires (the binge hook).
//   - episode table = E1-E5 kit verbatim (settling_in..harvest_festival, targets
//     300/600/1200/2400/600000, cfg_episode.json roster, ConfigValidator face 1) plus
//     the E6-E12 season continuation authored at T013 (demand-doc sec6 same-schema rows;
//     cumulative ledger now runs to 10,518,900 after E12).
//   - unlock gate = kit UnlockEpisode verbatim: E1 free (the pilot), episode N+1
//     hard-gated on episode N completion, idempotent - repeat and out-of-range rejects
//     answer with exactly one logged response and zero state mutation.
//   - climax math = kit verbatim: festival mult 512 (2^6 premium picks x 2 channel
```

```
//     x 4 festival), per-plot 51,200, E5 gain 614,400 exact (slogan promise);
//     cumulative ledger after E1..E5 = 300/900/2100/4500/618,900.
//   - resolve face = kit ResolveEpisode verbatim: a win marks completion and fires the
//     cliffhanger teaser plus one optional double-up offer; the 90% band is the
//     near-miss face (three optional rescue offers, zero forced ads); a deeper
//     shortfall is the clean-loss face. The offer EVENTS live here; the AdCapsule
//     policy (12/day hard cap, silent cap-reject) is the T009 seam and stays out.
//   - per-run re-arm = kit StartRun verbatim: a new run clears the unlock/completion
//     flags and the mult state, so E1 is free again on the next run.
// FieldMultiplier seam: FieldCore.FieldMultiplier is the episode-face yield driver
// (FieldCore T003 comment, "the episode face will drive yield later"); ApplyMultiplier
// writes the derived mult so field harvests land at 51,200 shi per plot at festival.
// Pure C#: zero UnityEngine, sim time is logical seconds set by the driver.
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class EpisodeChainEvent
    {
        public double t;
        public string type;
        public string detail;
        public EpisodeChainEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = det
    }
    public sealed class EpisodeCore
    {
        // ---- frozen timeline beats (kit Stage1 hook timeline, zero loosening) ----
        public const double EpisodeUnlockT = 5.50;  // episode 1 unlock (serialization teach beat)
        public const double CliffhangerT = 8.00;    // episode 1 complete -&gt; next-episode teaser fires
        // ---- frozen episode table (E1-E5 kit verbatim + E6-E12 T013 season continuation) ----
        // E1-E5 are the E4 kit roster frozen verbatim (zero loosening). E6-E12 are the
        // T013 season-continuation rows authored per demand-doc sec6 (same schema, new
        // authoring-frozen targets; cumulative ledger extends the kit ledger; the
        // reachability face rides the S2 numeric polish batch - ConfigValidator/EpisodeChainCheck
        // assertion faces re-cast in the T013 slice2 expansion, no gate wave before it).
        public const int EpisodeCount = 12;
        public static readonly string[] FrozenEpisodeIds =
        { "E1", "E2", "E3", "E4", "E5", "E6", "E7", "E8", "E9", "E10", "E11", "E12" };
        public static readonly string[] FrozenEpisodeNames =
        {
            "settling_in", "taking_root", "buying_fields", "digging_the_channel", "harvest_festival",
            "magistrate_arrival", "trade_road", "salt_iron_pact", "winter_rite",
            "spring_levee", "grand_gathering", "capital_envoy"
        };
        public static readonly long[] FrozenEpisodeTargets =
        { 300, 600, 1200, 2400, 600000, 750000, 1000000, 1250000, 1500000, 900000, 2000000, 2500000 };
        public static readonly long[] FrozenCumulative =
        { 300, 900, 2100, 4500, 618900, 1368900, 2368900, 3618900, 5118900, 6018900, 8018900, 10518900 }; // ledger afte
        // ---- frozen climax math (kit verbatim) ----
        public const int PremiumPicks = 6;      // the doubling lineup: six fertility-3 picks
```

```
public const int ChannelMult = 2;      // the irrigation channel doubles every harvest
public const int FestivalMult = 4;      // the harvest festival quadruples the estate
public const long FestivalMultDerived = 512; // 2^6 premium x 2 channel x 4 festival
public const long E5GainExact = 614400; // 12 plots x 51,200 (slogan promise)
public double SimTime;
public readonly List&lt;EpisodeChainEvent&gt; Events = new List&lt;EpisodeChainEvent&gt;();
public int RunCount;
public int EpisodesUnlocked;                        // count of legal unlocks
public readonly bool[] EpisodeUnlocked = new bool[EpisodeCount];
public readonly bool[] EpisodeCompleted = new bool[EpisodeCount];
public int EpisodeWins, EpisodeLosses, NearMisses;
public int RescueOffers, DoubleOffers;
public long EpisodeLedger;                          // cumulative grain credited this run
public long EpisodeStartGrain;                      // per-episode increment base
public bool ChannelBuilt;
public bool FestivalOn;
public readonly bool[] PremiumPlots = new bool[FieldCore.PlotCount];
public readonly string[] EpisodeIds = new string[EpisodeCount];
public readonly string[] EpisodeNames = new string[EpisodeCount];
public readonly long[] EpisodeTargets = new long[EpisodeCount];
public EpisodeCore(string[] episodeIds, string[] episodeNames, long[] episodeTargets)
{
    if (episodeIds == null || episodeIds.Length != EpisodeCount)
        throw new ArgumentException("episode table must carry exactly " + EpisodeCount + " rows");
    if (episodeNames == null || episodeNames.Length != EpisodeCount)
        throw new ArgumentException("episode table must carry a name for every episode");
    if (episodeTargets == null || episodeTargets.Length != EpisodeCount)
        throw new ArgumentException("episode table must carry a target for every episode");
    for (int i = 0; i &lt; EpisodeCount; i++)
    {
        if (episodeIds[i] != FrozenEpisodeIds[i])
            throw new ArgumentException("episode row " + i + " id '" + episodeIds[i] + "' != frozen '" + FrozenE
        if (episodeNames[i] != FrozenEpisodeNames[i])
            throw new ArgumentException("episode row " + i + " name '" + episodeNames[i] + "' != frozen '" + Fro
        if (episodeTargets[i] != FrozenEpisodeTargets[i])
            throw new ArgumentException("episode row " + i + " target " + episodeTargets[i] + " != frozen " + Fr
    }
    for (int i = 0; i &lt; EpisodeCount; i++)
    {
        EpisodeIds[i] = episodeIds[i];
        EpisodeNames[i] = episodeNames[i];
        EpisodeTargets[i] = episodeTargets[i];
    }
}
public void Log(string type, string detail) { Events.Add(new EpisodeChainEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
```

```
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
public int CountCompleted()
{
    int n = 0;
    for (int i = 0; i &lt; EpisodeCount; i++) if (EpisodeCompleted[i]) n++;
    return n;
}
// kit StartRun verbatim on the episode face: a new run re-arms the whole
// serialization state (unlock flags, completion flags, mult state, ledger).
public void StartRun(int runIdx)
{
    RunCount++;
    ChannelBuilt = false;
    FestivalOn = false;
    EpisodesUnlocked = 0;
    EpisodeLedger = 0;
    EpisodeStartGrain = 0;
    for (int i = 0; i &lt; EpisodeCount; i++) { EpisodeUnlocked[i] = false; EpisodeCompleted[i] = false; }
    for (int i = 0; i &lt; PremiumPlots.Length; i++) PremiumPlots[i] = false;
    Log("run_start", "run " + runIdx + ": the serial drama re-arms - episode 1 is free again");
}
// ==== serialization gate (kit UnlockEpisode verbatim): episode e unlocks only
// after episode e-1 completed. e is 1-based. E1 is free (the pilot). Every
// later episode is hard-gated. Repeated unlocks and out-of-range indices
// return false (idempotent gate, one logged response, zero mutation).
public bool UnlockEpisode(int e)
{
    if (e &lt; 1 || e &gt; EpisodeCount) { Log("episode_unlock_rejected", "out of range"); return false; }
    if (EpisodeUnlocked[e - 1]) { Log("episode_unlock_rejected", "already unlocked"); return false; }
    if (e &gt; 1 &amp;&amp; !EpisodeCompleted[e - 2]) { Log("episode_unlock_rejected", "previous episode not com
    EpisodeUnlocked[e - 1] = true;
    EpisodesUnlocked++;
    Log("episode_unlock", "episode " + e + " unlocked: " + EpisodeNames[e - 1]);
    return true;
}
// harvest income feeds the episode progress (wired by the presenter on harvest;
// drivers and checks call it directly). Non-positive credits are no-ops.
public void CreditGrain(long amount)
{
```

```
    if (amount &lt;= 0) return;
    EpisodeLedger += amount;
    Log("episode_credit", "+" + amount + " shi credited - episode ledger " + EpisodeLedger);
}
public void StartEpisode(int e)
{
    EpisodeStartGrain = EpisodeLedger;
    Log("episode_start", "episode " + e + " begins: " + EpisodeNames[e - 1]);
}
public long EpisodeGain() { return EpisodeLedger - EpisodeStartGrain; }
// kit ResolveEpisode verbatim: win fires the cliffhanger teaser and one
// optional double-up offer; the 90% band is the near-miss face (three rescue
// offers); anything deeper is a clean loss. Never a punishment (M05 honest
// failure face: the loss event, not the episode, carries the drama).
public bool ResolveEpisode(int e, long target)
{
    long gain = EpisodeGain();
    if (gain &gt;= target)
    {
        EpisodeWins++;
        EpisodeCompleted[e - 1] = true;
        Log("episode_won", "episode " + e + " clears the target");
        Cliffhanger(e);
        DoubleOffer();
        return true;
    }
    EpisodeLosses++;
    if ((double)gain &gt;= target * 0.9 - 1e-9)
    {
        NearMisses++;
        Log("near_miss", "so close - the episode fell just short, rescue offers pop up");
        RescueOffer();
    }
    else
    {
        Log("episode_lost_clean", "the episode falls short, the run ends here");
    }
    return false;
}
// Cliffhanger (kit verbatim): the binge hook - fires on every episode win.
public void Cliffhanger(int e)
{
    if (e &lt; EpisodeCount)
        Log("cliffhanger_fire", "next-episode teaser: the county magistrate rides in - her smile hides a blade")
    else
        Log("cliffhanger_fire", "season finale teaser: the frontier estate stands - next season, the capital cal
}
// Near-miss rescue (kit verbatim): three optional rewarded-video outs - the
// player may take one, never must (zero forced ads, ads=0 E4 anchor).
public void RescueOffer()
```

```
{
    Log("rescue_offer", "optional video: revive and retry this episode?");
    Log("rescue_offer", "optional video: replant the failed plots?");
    Log("rescue_offer", "optional video: double the granary to pass?");
    RescueOffers += 3;
}
public void DoubleOffer()
{
    DoubleOffers++;
    Log("double_offer", "optional video: double this episode harvest?");
}
// ==== mult state (kit verbatim) ====
public void BuildChannel()
{
    ChannelBuilt = true;
    Log("channel_build", "the irrigation channel is dug - every harvest doubles");
}
public void PickPremiumPlot(int idx)
{
    if (idx &lt; 0 || idx &gt;= PremiumPlots.Length) return;
    PremiumPlots[idx] = true;
    Log("premium_pick", "plot " + idx + " is coaxed to top fertility - it joins the doubling lineup");
}
public void StartFestival()
{
    FestivalOn = true;
    Log("festival_on", "the harvest festival begins - the whole estate answers");
}
// kit EpisodeMult verbatim: 1 base, x2 once the channel is built, and at the
// festival the premium lineup (2^6 with six picks) and the festival itself
// (x4) stack -&gt; 512.
public long EpisodeMult()
{
    long m = 1;
    if (ChannelBuilt) m *= ChannelMult;
    if (FestivalOn)
    {
        int premium = 0;
        for (int i = 0; i &lt; PremiumPlots.Length; i++) if (PremiumPlots[i]) premium++;
        m *= (1L &lt;&lt; premium);
        m *= FestivalMult;
    }
    return m;
}
// FieldMultiplier seam (T003 comment): the episode face writes the derived
// mult into the field, so a festival harvest lands at 51,200 shi per plot.
public void ApplyMultiplier(FieldCore field)
{
    if (field == null) return;
    field.FieldMultiplier = EpisodeMult();
```

```
        }
    }
}
// EstateCore.cs
// EstateCore.cs - S1 T004 estate five-stage system core for ExileFarm (B1269 kit-port).
// Ported under the kit-port law: every frozen value stays verbatim - zero new tuning.
// Frozen anchors (all traceable to the E4 kit, Design/evidence/m6_exilefarm_graybox_kit/):
//   - five stages = requirement M04 ladder reed shed -&gt; rammed-earth courtyard -&gt;
//     brick-tile house -&gt; frontier trading post -&gt; first manor of the exile land
//     (i18n keys estate_stage_1..5 shipped in the closed 28-key set, T002 slice2)
//   - upgrade_cost_shi = kit episode targets verbatim 300/600/1200/2400/600000.
//     Derivation: the core loop walks one ladder - gold -&gt; upgrade the estate -&gt;
//     progress bar full -&gt; next episode - so each estate stage's standard IS the
//     matching episode's frozen target (cfg_episode.json, ConfigValidator face 1).
//   - unlock_plots = E4 per-episode plot gains 3/6/12 fields (kit README "e1=300 (3
//     fields) / e2=600 (6 fields) / e3=1200 (12 fields)") capped at PlotCount 12.
//   - starter_plots = 2, kit StarterPlotsT 0.50s "two starter plots granted": the
//     exiled wife tills her first two plots before the reed shed exists (E1 goal
//     chain: till first -&gt; harvest first -&gt; THEN raise the shed).
// Gate semantics (kit-verbatim): threshold semantics, NOT spend - the granary is
// the cumulative income ledger single source (kit replay identity 10,534,800) and
// goal_3 "fill the granary to 1000 shi" requires the granary to keep filling, so
// the estate gate READS the cumulative ledger against the stage standard
// ("gold reaches the standard -&gt; tap to upgrade") and never consumes it. With the
// frozen costs the ladder lands inside its matching episode every time (cumulative
// ledger 300/900/2100/4500/618900 crosses 300/600/1200/2400/600000 mid-E1..E5).
// Serial walk: UpgradeEstate takes no stage argument, so stages can never be
// skipped - the same hard-serial shape as the kit UnlockEpisode gate.
// Ticket face: stage unlocks field slots (requirement M02 "slots unlock by estate
// level"); the episode yield multipliers stay on the FieldMultiplier seam (T005).
// Pure C#: zero UnityEngine, sim time is logical seconds set by the driver.
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class EstateEvent
    {
        public double t;
        public string type;
        public string detail;
        public EstateEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public sealed class EstateCore
    {
        // ---- frozen estate table (kit verbatim, zero loosening) ----
        public const int EstateStageCount = 5;
        public const int StarterPlots = 2;   // kit StarterPlotsT: two starter plots granted at 0.50s
        public static readonly string[] FrozenStageIds =
        {
            "reed_shed", "rammed_earth_courtyard", "brick_tile_house",
```

```
    "frontier_trading_post", "first_manor_of_the_exile_land"
};
public static readonly long[] FrozenStageCosts = { 300, 600, 1200, 2400, 600000 }; // kit episode targets
public static readonly int[] FrozenStageUnlockPlots = { 3, 6, 12, 12, 12 };        // E4 plot ladder
public double SimTime;
public readonly List&lt;EstateEvent&gt; Events = new List&lt;EstateEvent&gt;();
public int Stage;                       // 0 = unsettled (no estate yet), 1..5 = the five stages
public long EstateLedger;               // cumulative grain credited (threshold source, never spent)
public int UpgradesTotal;
public readonly string[] StageIds = new string[EstateStageCount];
public readonly long[] StageCosts = new long[EstateStageCount];
public readonly int[] StageUnlockPlots = new int[EstateStageCount];
public readonly int StarterPlotCount;
public EstateCore(string[] stageIds, long[] stageCosts, int[] stageUnlockPlots, int starterPlots)
{
    if (stageIds == null || stageIds.Length != EstateStageCount)
        throw new ArgumentException("estate table must carry exactly " + EstateStageCount + " stage rows");
    if (stageCosts == null || stageCosts.Length != EstateStageCount)
        throw new ArgumentException("estate table must carry a cost row for every stage");
    if (stageUnlockPlots == null || stageUnlockPlots.Length != EstateStageCount)
        throw new ArgumentException("estate table must carry an unlock row for every stage");
    for (int i = 0; i &lt; EstateStageCount; i++)
    {
        if (stageIds[i] != FrozenStageIds[i])
            throw new ArgumentException("stage row " + i + " '" + stageIds[i] + "' != frozen '" + FrozenStageIds
        if (stageCosts[i] != FrozenStageCosts[i])
            throw new ArgumentException("stage row " + i + " cost " + stageCosts[i] + " != frozen " + FrozenStag
        if (stageUnlockPlots[i] != FrozenStageUnlockPlots[i])
            throw new ArgumentException("stage row " + i + " unlock_plots " + stageUnlockPlots[i] + " != frozen
    }
    if (starterPlots != StarterPlots)
        throw new ArgumentException("starter_plots " + starterPlots + " != frozen " + StarterPlots + " (kit Star
    for (int i = 0; i &lt; EstateStageCount; i++)
    {
        StageIds[i] = stageIds[i];
        StageCosts[i] = stageCosts[i];
        StageUnlockPlots[i] = stageUnlockPlots[i];
    }
    StarterPlotCount = starterPlots;
}
public void Log(string type, string detail) { Events.Add(new EstateEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
```

```
            return false;
        }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
            return n;
        }
        // harvest income feeds the estate threshold (wired by the presenter on harvest;
        // drivers and checks call it directly). Non-positive credits are no-ops.
        public void CreditGrain(long amount)
        {
            if (amount &lt;= 0) return;
            EstateLedger += amount;
            Log("estate_credit", "+" + amount + " shi credited - estate ledger " + EstateLedger);
        }
        // ticket face: how many field slots the current estate stage opens
        public int UnlockedPlots
        {
            get { return Stage == 0 ? StarterPlotCount : StageUnlockPlots[Stage - 1]; }
        }
        public bool IsPlotSlotOpen(int idx)
        {
            return idx &gt;= 0 &amp;&amp; idx &lt; UnlockedPlots;
        }
        // cost of the next stage; -1 at the cap
        public long NextCost()
        {
            return Stage &lt; EstateStageCount ? StageCosts[Stage] : -1;
        }
        // kit-verbatim reject semantics: one logged response per invalid tap, zero state change
        public bool UpgradeEstate()
        {
            if (Stage &gt;= EstateStageCount)
            {
                Log("estate_upgrade_rejected", "already the first manor of the exile land");
                return false;
            }
            if (EstateLedger &lt; StageCosts[Stage])
            {
                Log("estate_upgrade_rejected", "granary " + EstateLedger + " does not reach the " + StageIds[Stage] + "
                return false;
            }
            Stage++;
            UpgradesTotal++;
            Log("estate_upgrade", "the " + StageIds[Stage - 1] + " is raised - stage " + Stage + "/" + EstateStageCount)
            Log("slots_open", UnlockedPlots + " field slots answer the estate");
            return true;
        }
    }
```

```
}
// EventCore.cs
// EventCore.cs - S2 T012 comedy event engine for ExileFarm (B1836 kit-port).
// Ported from the G08 CampusEventEngine (the series-shared SYS04 family asset,
// CrazyCampus T006 slice A) under the kit-port law: the pacing baselines, the
// episode-scoped pool filter, the rotation cursor, the chain resolver and the
// save/restore faces stay verbatim - only the fx shape and the chain families
// are ExileFarm's own. Pure data face with an injected clock: the presenter
// walks it with the same virtual seconds the cores ride, headless gates drive
// it directly. Zero RNG (the pool cursor is a deterministic rotation), zero
// wall-clock, zero server faces (red line 12). Pacing: the first card lands
// inside the &lt;=10s red-line window, later cards every 3-5 minutes of play;
// chained cards book their comedy follow-up beats (the beast night ends with
// the walls standing, the harvest fair walks its three-step dance). Chain
// beats are pure telemetry - the card fx already carries the net effect, so
// the grain ledger stays exact. The loss family rides the M07 law: a loss
// card is an emotion beat with a grain fx only - the engine has no dex wipe
// call anywhere, the land/crop dex keeps everything (CodexCore structural
// face, re-verified by EventCheck). The G08 affinity slot has no ExileFarm
// counterpart (no goodwill stat in the frozen economy) - structural
// omission, zero threshold change. English comedy prose only (the i18n face
// lands T015 with the full event text family).
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public enum FarmEventKind { Card, ChainStep }
    /// &lt;summary&gt;One logged event beat: the card itself or a chain step.&lt;/summary&gt;
    public sealed class FarmEventLogEntry
    {
        public double t;
        public string id = "";            // EV01..EV06
        public FarmEventKind kind;
        public string step = "";          // chain step tag, "" on the card itself
        public long grainFx;              // the card effect (chain steps carry zero)
    }
    /// &lt;summary&gt;One comedy event card: episode-scoped, family-typed, a small
    /// signed grain correction and an optional chain descriptor. Pure data.&lt;/summary&gt;
    public sealed class FarmEventDef
    {
        public string id = "";
        public int episode;          // 1-5 scope, 0 = any episode
        public string family = "";   // "loss" / "joy"
        public long grainFx;         // signed shi delta: loss &lt; 0, joy &gt; 0
        public string chain = "";    // chain descriptor, "" = none
        public string detail = "";   // English comedy prose (i18n face lands T015)
    }
    public sealed class EventCore
    {
        // pacing baselines (G08 verbatim, the only-up law: may tighten, never loosen)
        public const double FirstEventDelay = 8.00;    // first card inside the &lt;=10s red line
```

```
public const double EventCadenceSeconds = 240;  // later cards: one per 3-5 min band
public const double CadenceBandMin = 180;
public const double CadenceBandMax = 300;
public const double ChainStepDelay = 4.00;      // chain beats trail their card
// frozen S2 starter table faces
public const int EventCount = 6;         // EV01-EV06 (requirement sec 6 starter six)
public const int LossFamilyCount = 3;    // frost / beast pack / shed wind
public const int JoyFamilyCount = 3;     // noble passes / harvest fair / caravan egg
public const int EventChainCount = 2;    // EV02 wall_up + EV05 county_fair
public const long EventFxMaxAbsShi = 500; // event effects stay small corrections
public const int EpisodeScopeCount = 5;  // kit EpisodeCount mirror (E1-E5)
readonly FarmEventDef[] table;
readonly List&lt;FarmEventLogEntry&gt; log = new List&lt;FarmEventLogEntry&gt;();
readonly List&lt;ChainBeat&gt; pending = new List&lt;ChainBeat&gt;();
int episode;
int fired;
int cursor;
long grainBooked;
double simTime;
double nextCardAt;
struct ChainBeat
{
    public double at;
    public string id;
    public string step;
}
/// &lt;summary&gt;Series wiring face: inject any frozen event table and the
/// starting episode. The farm default wires the S2 starter table.&lt;/summary&gt;
public EventCore(FarmEventDef[] eventTable, int startEpisode)
{
    table = eventTable ?? new FarmEventDef[0];
    episode = startEpisode &lt; 1 ? 1 : (startEpisode &gt; EpisodeScopeCount ? EpisodeScopeCount : startEpisode)
    nextCardAt = FirstEventDelay;
}
/// &lt;summary&gt;The frozen S2 starter table (six comedy cards, three loss
/// sources and three joy sources, requirement sec 6 event table).&lt;/summary&gt;
public static FarmEventDef[] StarterTable()
{
    return new[]
    {
        new FarmEventDef { id = "EV01", episode = 0, family = "loss", grainFx = -200, chain = "",
            detail = "a night frost tiptoes through the field - the seedlings shiver, the granary coughs up 200
        new FarmEventDef { id = "EV02", episode = 0, family = "loss", grainFx = -300, chain = "wall_up",
            detail = "the night beast pack sniffs at the walls - the walls hold, the toll is paid in grain" },
        new FarmEventDef { id = "EV03", episode = 0, family = "loss", grainFx = -150, chain = "",
            detail = "a big wind lifts the reed-shed roof - it sails three plots over and lands good as new" },
        new FarmEventDef { id = "EV04", episode = 0, family = "joy", grainFx = 400, chain = "",
            detail = "a kind noble passes the frontier and leaves a pouch of grain with a wink" },
        new FarmEventDef { id = "EV05", episode = 0, family = "joy", grainFx = 500, chain = "county_fair",
            detail = "the harvest fair rolls in - lanterns, dumplings, the whole estate dances" },
```

```
            new FarmEventDef { id = "EV06", episode = 0, family = "joy", grainFx = 250, chain = "",
                detail = "the trade caravan tucks a lucky egg into the seed sacks - it hatches into grain" },
        };
    }
    public static EventCore ForFarm(int startEpisode)
    {
        return new EventCore(StarterTable(), startEpisode);
    }
    public int Episode { get { return episode; } }
    public int Fired { get { return fired; } }
    public long GrainBooked { get { return grainBooked; } }
    public double SimTime { get { return simTime; } }
    public double NextCardAt { get { return nextCardAt; } }
    public int CursorFace { get { return cursor; } }
    public IList&lt;FarmEventLogEntry&gt; Log { get { return log; } }
    /// &lt;summary&gt;Read face over the in-flight chain beats (save capture
    /// needs the pending tail, the log alone cannot rebuild it).&lt;/summary&gt;
    public ChainBeatInfo[] PendingBeats()
    {
        var a = new ChainBeatInfo[pending.Count];
        for (int i = 0; i &lt; pending.Count; i++)
        {
            a[i].at = pending[i].at;
            a[i].id = pending[i].id;
            a[i].step = pending[i].step;
        }
        return a;
    }
    /// &lt;summary&gt;Public copy of the pending chain beat (save wire face).&lt;/summary&gt;
    public struct ChainBeatInfo
    {
        public double at;
        public string id;
        public string step;
    }
    /// &lt;summary&gt;Save-restore factory (S05 save.events): rebuilds the engine
    /// at an exact captured state - rotation cursor, fired count, grain
    /// ledger, clock, next-card grid, the whole card log and the pending
    /// chain tail - so resumed runs keep dealing the same cards.&lt;/summary&gt;
    public static EventCore Restore(FarmEventDef[] eventTable, int savedEpisode,
        int savedFired, int savedCursor, long savedGrain, double savedSimTime,
        double savedNextCardAt, FarmEventLogEntry[] savedLog, ChainBeatInfo[] savedPending)
    {
        var e = new EventCore(eventTable, savedEpisode);
        e.fired = savedFired &lt; 0 ? 0 : savedFired;
        e.cursor = savedCursor &lt; 0 ? 0 : savedCursor;
        e.grainBooked = savedGrain;
        e.simTime = savedSimTime;
        e.nextCardAt = savedNextCardAt;
        if (savedLog != null)
```

```
        for (int i = 0; i &lt; savedLog.Length; i++)
            e.log.Add(savedLog[i]);
    if (savedPending != null)
        for (int i = 0; i &lt; savedPending.Length; i++)
            e.pending.Add(new ChainBeat
            {
                at = savedPending[i].at,
                id = savedPending[i].id,
                step = savedPending[i].step,
            });
    return e;
}
/// &lt;summary&gt;Episode moves re-scope the pool live; the rotation cursor
/// keeps running so a mid-season player never sees the pool restart.&lt;/summary&gt;
public void SetEpisode(int ep)
{
    episode = ep &lt; 1 ? 1 : (ep &gt; EpisodeScopeCount ? EpisodeScopeCount : ep);
}
/// &lt;summary&gt;The episode-scoped pool: the episode's own rows plus the
/// any-episode rows, in frozen table order (deterministic).&lt;/summary&gt;
public FarmEventDef[] Pool()
{
    var pool = new List&lt;FarmEventDef&gt;();
    for (int i = 0; i &lt; table.Length; i++)
        if (table[i].episode == episode || table[i].episode == 0) pool.Add(table[i]);
    return pool.ToArray();
}
FarmEventDef DefById(string id)
{
    for (int i = 0; i &lt; table.Length; i++)
        if (table[i].id == id) return table[i];
    return null;
}
/// &lt;summary&gt;Clock tick: due chain beats land first (they trail their
/// card), then any due card fires and books its own follow-ups. Big
/// dt jumps drain every due beat in schedule order - deterministic.&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt; 0) dt = 0;
    double target = simTime + dt;
    bool progressed = true;
    while (progressed)
    {
        progressed = false;
        double nextChainAt = double.MaxValue;
        for (int i = 0; i &lt; pending.Count; i++)
        {
            if (pending[i].at &lt;= target)
            {
                var beat = pending[i];
```

```
                pending.RemoveAt(i);
                i--;
                log.Add(new FarmEventLogEntry
                {
                    t = beat.at, id = beat.id, kind = FarmEventKind.ChainStep, step = beat.step
                });
                progressed = true;
            }
            else if (pending[i].at &lt; nextChainAt) nextChainAt = pending[i].at;
        }
        if (nextCardAt &lt;= target &amp;&amp; Pool().Length &gt; 0)
        {
            FireCard(target);
            progressed = true;
        }
    }
    simTime = target;
}
void FireCard(double target)
{
    var pool = Pool();
    var def = pool[cursor % pool.Length];
    cursor++;
    fired++;
    double at = nextCardAt &lt;= target ? nextCardAt : target;
    log.Add(new FarmEventLogEntry { t = at, id = def.id, kind = FarmEventKind.Card, grainFx = def.grainFx });
    grainBooked += def.grainFx;
    if (def.chain.Length &gt; 0) ScheduleChain(def, at);
    nextCardAt = at + EventCadenceSeconds;
}
/// &lt;summary&gt;Chain law: the descriptor names the family. The wall family
/// ends the beast night with the walls standing; the fair family walks
/// the three-step harvest dance. Any other descriptor fails loud at
/// schedule time - never silently dropped.&lt;/summary&gt;
void ScheduleChain(FarmEventDef def, double cardAt)
{
    string[] steps;
    if (def.chain.Contains("wall_up"))
    {
        steps = new[] { "beasts_retreat", "walls_stand" };
    }
    else if (def.chain.Contains("county_fair"))
    {
        steps = new[] { "lanterns_lit", "dumplings_shared", "encore_begins" };
    }
    else
    {
        throw new System.InvalidOperationException("unknown chain family: " + def.chain);
    }
    for (int i = 0; i &lt; steps.Length; i++)
```

```
                pending.Add(new ChainBeat
                {
                    at = cardAt + (i + 1) * ChainStepDelay,
                    id = def.id,
                    step = steps[i]
                });
        }
        public double TimeOfCard(string id)
        {
            for (int i = 0; i &lt; log.Count; i++)
                if (log[i].kind == FarmEventKind.Card &amp;&amp; log[i].id == id) return log[i].t;
            return -1.0;
        }
        public int CountCards(string id)
        {
            int n = 0;
            for (int i = 0; i &lt; log.Count; i++)
                if (log[i].kind == FarmEventKind.Card &amp;&amp; log[i].id == id) n++;
            return n;
        }
        public int CountCards()
        {
            int n = 0;
            for (int i = 0; i &lt; log.Count; i++)
                if (log[i].kind == FarmEventKind.Card) n++;
            return n;
        }
        /// &lt;summary&gt;Deterministic log fingerprint - the double-run byte-parity
        /// face (zero-RNG family): identical drives land on identical bytes.&lt;/summary&gt;
        public string EventSig()
        {
            var sb = new System.Text.StringBuilder();
            for (int i = 0; i &lt; log.Count; i++)
            {
                var e = log[i];
                sb.Append(e.id).Append(e.kind == FarmEventKind.Card ? "#card" : "#step:")
                  .Append(e.step).Append('@').Append(e.t.ToString("0.###")).Append(';');
            }
            sb.Append("fired=").Append(fired).Append("|cursor=").Append(cursor)
              .Append("|grain=").Append(grainBooked);
            return sb.ToString();
        }
    }
}
// FieldCore.cs
// FieldCore.cs - S1 T003 field system core for ExileFarm (B1266 kit-port).
// Ported from the E4 graybox kit (Design/evidence/m6_exilefarm_graybox_kit/GrayboxSim.cs)
// under the kit-port law: every E4 frozen value stays verbatim - first till 1.00s,
// first plant triple 1.50s, first harvest 2.50s. Zero new tuning.
// Structural S1 extension vs the kit: the Ripe state + a table-driven growth gate
```

```
// (requirement M02 "plant -&gt; auto-grow per table"; the graybox skipped growth because
// growth was out of E4 scope). The growth table is injected (cfg_field.json face) so
// balancing stays data-only. Growth seconds frozen at 1.00 = the E4 Stage1 derivation
// first harvest 2.50 = first plant 1.50 + growth 1.00.
// Scope = field face only (till / plant / grow / harvest + crop rotation back to
// Tilled). Episode multipliers, goals, dex and share faces belong to T005-T008; the
// FieldMultiplier seam (default 1) is where the episode face will drive yield later.
// Pure C#: zero UnityEngine, sim time is logical seconds set by the driver (check/bot).
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class FieldEvent
    {
        public double t;
        public string type;
        public string detail;
        public FieldEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public enum PlotState { Wild = 0, Tilled, Planted, Ripe }
    public sealed class FieldCore
    {
        // ---- E4 frozen hook timeline, field face (kit verbatim, zero loosening) ----
        public const double FirstTillT = 1.00;     // first till (zero-learning-cost 3s hook)
        public const double FirstPlantT = 1.50;    // first plant: tap + sfx + grain pop same tick
        public const double FirstHarvestT = 2.50;  // first harvest = 1.50 + growth 1.00 (derived-exact)
        public const double FrozenGrowthSeconds = 1.00; // every crop row: the E4 first-harvest anchor
        // ---- frozen economy constants (kit verbatim) ----
        public const int PlotCount = 12;
        public const long BaseYield = 100;         // shi per plot per harvest at multiplier 1
        public const int CropCount = 3;
        public static readonly string[] CropNames = { "millet", "bean", "wheat" };
        public static readonly string[] PlotTerrainNames =
        {
            "wild_steppe", "sunny_slope", "river_bend", "rocky_ridge", "marsh_hollow", "windy_knoll",
            "old_orchard", "bamboo_edge", "well_side", "south_terrace", "north_field", "cliff_ledge"
        };
        public double SimTime;
        public readonly List&lt;FieldEvent&gt; Events = new List&lt;FieldEvent&gt;();
        public readonly PlotState[] Plots = new PlotState[PlotCount];
        public readonly int[] PlotTerrain = new int[PlotCount];
        public readonly double[] PlantedAt = new double[PlotCount];
        public readonly double[] RipeAt = new double[PlotCount];
        public readonly int[] PlotCrop = new int[PlotCount];
        public readonly double[] CropGrowthSeconds = new double[CropCount];
        public long FieldMultiplier = 1;           // episode face seam (T005), default 1
        public long GrainTotal;                    // field ledger single source
        public int TillsTotal, PlantsTotal, HarvestsTotal;
        public FieldCore(string[] cropNames, double[] growthSeconds, string[] terrainRoster)
        {
```

```
//     and FirstGoalDoneAt, so the chain replays from the first goal every run.
// Structural S1 extension vs the kit: the GoalDoneAt[] per-goal timestamps (the
// kit recorded only the first goal time) - the production window face needs every
// goal's landing time to assert the M06 per-goal window. Zero threshold change.
// State consumption: the chain reads the field counters (tills / harvests / grain
// ledger) and the episode face (unlocks / festival) through the real cores - the
// presenter calls BeatGoals after each till / plant / harvest / unlock / festival
// mutation (the five kit call sites). Pure C#: zero UnityEngine.
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class GoalChainEvent
    {
        public double t;
        public string type;
        public string detail;
        public GoalChainEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail
    }
    public sealed class GoalCore
    {
        // ---- frozen beats (kit six-beat table M06 face, zero loosening) ----
        public const double FirstGoalBeatT = 2.50;     // first narrative goal done (bot opening: third till at 2.50s)
        public const double GoalWindowSeconds = 60.0;  // every goal must land within a 60s window (kit first-goal face)
        // ---- frozen goal table (kit GoalTexts verbatim) ----
        public const int GoalCount = 5;
        public static readonly string[] FrozenGoalTexts =
        {
            "till 3 plots of wild land", "bring in the first harvest", "fill the granary to 1000 shi",
            "unlock the next episode", "hold the harvest festival"
        };
        // ---- frozen predicate table (kit BeatGoals switch verbatim) ----
        public const int Goal0Tills = 3;          // goal 1: till three plots of wild land
        public const int Goal1Harvests = 1;       // goal 2: bring in the first harvest
        public const long Goal2Grain = 1000;      // goal 3: fill the granary to 1000 shi
        public const int Goal3Episodes = 2;       // goal 4: unlock the next episode
        public double SimTime;
        public readonly List&lt;GoalChainEvent&gt; Events = new List&lt;GoalChainEvent&gt;();
        public int CurrentGoal;
        public double FirstGoalDoneAt = -1.0;                    // kit field verbatim
        public readonly double[] GoalDoneAt = new double[GoalCount]; // S1 window face: per-goal landing times (-1 = not
        public readonly string[] GoalTexts = new string[GoalCount];
        public GoalCore(string[] goalTexts)
        {
            if (goalTexts == null)
                throw new ArgumentException("goal table must carry exactly " + GoalCount + " rows");
            if (goalTexts.Length != GoalCount)
                throw new ArgumentException("goal table must carry exactly " + GoalCount + " rows, got " + goalTexts.Len
            for (int i = 0; i &lt; GoalCount; i++)
            {
```

```
        if (goalTexts[i] != FrozenGoalTexts[i])
            throw new ArgumentException("goal row " + i + " '" + goalTexts[i] + "' != frozen '" + FrozenGoalText
    }
    for (int i = 0; i &lt; GoalCount; i++)
    {
        GoalTexts[i] = goalTexts[i];
        GoalDoneAt[i] = -1.0;
    }
}
public void Log(string type, string detail) { Events.Add(new GoalChainEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
// kit StartRun goal face verbatim: a new run re-arms the chain from the
// first goal (plus the S1 window face reset). The caller owns SimTime.
public void StartRun()
{
    CurrentGoal = 0;
    FirstGoalDoneAt = -1.0;
    for (int i = 0; i &lt; GoalCount; i++) GoalDoneAt[i] = -1.0;
}
// ==== narrative goal chain: auto-advance, same tick (kit BeatGoals verbatim).
// The five kit call sites: till / plant / harvest / unlock / festival. The
// predicates read the real cores; a satisfied goal completes and fires the
// next goal on the same tick, cascading while the loop still advances.
public void BeatGoals(FieldCore field, EpisodeCore episode)
{
    if (field == null || episode == null) return;
    bool advanced = true;
    while (advanced &amp;&amp; CurrentGoal &lt; GoalCount)
    {
        advanced = false;
        switch (CurrentGoal)
        {
            case 0:
                if (field.TillsTotal &gt;= Goal0Tills) { GoalDone(); advanced = true; }
```

```
                        break;
                    case 1:
                        if (field.HarvestsTotal &gt;= Goal1Harvests) { GoalDone(); advanced = true; }
                        break;
                    case 2:
                        if (field.GrainTotal &gt;= Goal2Grain) { GoalDone(); advanced = true; }
                        break;
                    case 3:
                        if (episode.EpisodesUnlocked &gt;= Goal3Episodes) { GoalDone(); advanced = true; }
                        break;
                    case 4:
                        if (episode.FestivalOn) { GoalDone(); advanced = true; }
                        break;
                }
            }
        }
        // kit GoalDone verbatim (plus the S1 window-face timestamp): first-goal time
        // recorded once, one goal_complete log, the next goal fires on the same tick.
        void GoalDone()
        {
            if (FirstGoalDoneAt &lt; 0 &amp;&amp; CurrentGoal == 0) FirstGoalDoneAt = SimTime;
            if (GoalDoneAt[CurrentGoal] &lt; 0) GoalDoneAt[CurrentGoal] = SimTime;
            Log("goal_complete", "goal done: " + GoalTexts[CurrentGoal]);
            CurrentGoal++;
            if (CurrentGoal &lt; GoalCount)
                Log("goal_fire", "next goal: " + GoalTexts[CurrentGoal]);
        }
    }
}
// GrayboxBot.cs
// GrayboxBot.cs - naive bare-play bot for ExileFarm E4 graybox (M6-B lane).
// Runs 20 short runs of 5 serialized episodes each on frozen deterministic
// episode templates. Most runs clear all five episodes on the scripted line
// (till -&gt; plant -&gt; harvest -&gt; premium pick -&gt; next episode unlock). Runs r%7==3
// hit a hard target on episode 4 (gain lands inside the 0.9x..1.0x near-miss band)
// -&gt; three rescue offers pop up, the bot declines all of them (zero ads), the run
// ends honestly, and the land/crop dex keeps everything (zero punishment).
// Premium picks after episodes 1-3 build the 2^6 doubling lineup; the channel
// built inside episode 4 doubles every harvest; the episode-5 festival stacks
// x4 on top -&gt; climax mult 512, harvest 614,400 shi exact (slogan promise).
// Zero RNG by construction; crop rotation lets plots replant every episode.
using System;
using System.Collections.Generic;
using Sy.ExileFarm;
namespace Sy.ExileFarm
{
    public sealed class GrayboxBot
    {
        private readonly GrayboxSim sim;
        public int Runs;
```

```
public readonly List&lt;double&gt; FirstGoalAt = new List&lt;double&gt;();
public int UnlockTrueCount, UnlockFalseCount;
public GrayboxBot(GrayboxSim sim) { this.sim = sim; }
void Step() { sim.SimTime += 0.50; }
void ActTill(int idx) { sim.TillPlot(idx); Step(); }
void ActPlant(int idx, int crop) { sim.PlantPlot(idx, crop); Step(); }
void ActHarvest(int idx) { sim.HarvestPlot(idx); Step(); }
void PlantRow(int from, int to, int crop)
{
    for (int i = from; i &lt;= to; i++) ActPlant(i, crop);
}
void HarvestRow(int from, int to)
{
    for (int i = from; i &lt;= to; i++) ActHarvest(i);
}
void TillRow(int from, int to)
{
    for (int i = from; i &lt;= to; i++) ActTill(i);
}
void UnlockStart(int e)
{
    bool ok = sim.UnlockEpisode(e);
    if (ok) UnlockTrueCount++; else UnlockFalseCount++;
    Step();
    sim.StartEpisode(e);
    Step();
}
// Episode templates (frozen at E4 authoring): per-episode increment targets
// e1 300 / e2 600 / e3 1200 / e4 2400 (hard variant 2640 -&gt; near-miss) /
// e5 climax 600,000 vs actual 614,400 (mult 512 = 2^6 premium x 2 channel x 4 festival).
public void Run(int runs)
{
    for (int r = 0; r &lt; runs; r++)
    {
        Runs++;
        sim.SimTime = 0.0;
        sim.StartRun(r);
        Step();
        bool alive = true;
        // episode 1: settle in - till 3, plant 3, harvest 300
        UnlockStart(1);
        TillRow(0, 2);
        PlantRow(0, 2, (r + 1) % 3);
        HarvestRow(0, 2);
        alive = sim.ResolveEpisode(1, GrayboxSim.EpisodeTargets[0]);
        Step();
        if (alive) { sim.PickPremiumPlot(0); sim.PickPremiumPlot(1); Step(); }
        if (!alive) { sim.RunLost(r); FirstGoalAt.Add(sim.FirstGoalDoneAt); continue; }
        // episode 2: take root - till 3 more, plant 6, harvest 600
        UnlockStart(2);
```

```
                TillRow(3, 5);
                PlantRow(0, 5, (r + 2) % 3);
                HarvestRow(0, 5);
                alive = sim.ResolveEpisode(2, GrayboxSim.EpisodeTargets[1]);
                Step();
                if (alive) { sim.PickPremiumPlot(2); sim.PickPremiumPlot(3); Step(); }
                if (!alive) { sim.RunLost(r); FirstGoalAt.Add(sim.FirstGoalDoneAt); continue; }
                // episode 3: buy fields - till all 12, plant 12, harvest 1200
                UnlockStart(3);
                TillRow(6, 11);
                PlantRow(0, 11, (r + 3) % 3);
                HarvestRow(0, 11);
                alive = sim.ResolveEpisode(3, GrayboxSim.EpisodeTargets[2]);
                Step();
                if (alive) { sim.PickPremiumPlot(4); sim.PickPremiumPlot(5); Step(); }
                if (!alive) { sim.RunLost(r); FirstGoalAt.Add(sim.FirstGoalDoneAt); continue; }
                // episode 4: dig the channel - every harvest doubles -&gt; 2400
                bool hard = (r % 7 == 3);
                UnlockStart(4);
                sim.BuildChannel();
                Step();
                PlantRow(0, 11, (r + 4) % 3);
                HarvestRow(0, 11);
                alive = sim.ResolveEpisode(4, hard ? GrayboxSim.Episode3HardTarget : GrayboxSim.EpisodeTargets[3]);
                Step();
                if (!alive) { sim.RunLost(r); FirstGoalAt.Add(sim.FirstGoalDoneAt); continue; }
                // episode 5: harvest festival - mult 512, gain 614,400 exact
                UnlockStart(5);
                sim.StartFestival();
                Step();
                PlantRow(0, 11, (r + 5) % 3);
                HarvestRow(0, 11);
                alive = sim.ResolveEpisode(5, GrayboxSim.EpisodeTargets[4]);
                Step();
                if (!alive) { sim.RunLost(r); FirstGoalAt.Add(sim.FirstGoalDoneAt); continue; }
                sim.RunWon();
                Step();
                if (sim.TerrainDex.Count &gt;= 8) sim.ShareCard();   // episode achievement card, never a score form
                FirstGoalAt.Add(sim.FirstGoalDoneAt);
            }
        }
    }
}
// GrayboxSim.cs
// GrayboxSim.cs - pure C# deterministic logic prototype for ExileFarm (E4 graybox, M6-B lane).
// Ancient-exile frontier farming: exiled to a border wasteland, till wild plots, plant,
// harvest, grow the frontier estate, and unlock the serial drama episodes (season
// serialization = the structure increment vs the idle-farm loop base). Episode N+1 is
// hard-gated on episode N completion; every episode win fires a next-episode teaser
// cliffhanger (the binge hook). Short-run IAA shape: near-miss rescue offers (revive /
```

```
// replant / double - purely optional rewarded-video points, never required, the loop
// closes with zero ads), an optional double-up offer after each won episode. The land
// dex is the zero-punishment meta face: a lost run never wipes the dex.
// IP guardrail (U033 / B1227 admission): the source drama's title and the "guilty
// wife" wording are BANNED from every face - the trope is used, the IP is not.
// Share face (B904 guardrail #2 analog): the share card is an episode achievement /
// land-dex card with an X032-style code; score-form result display stays out of the
// share payload. No UnityEngine dependency: sim time is logical seconds; wall-clock
// perf is measured by the editor check. Zero RNG by construction.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.ExileFarm
{
    public sealed class SimEvent
    {
        public double t;
        public string type;
        public string detail;
        public SimEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public sealed class GrayboxSim
    {
        // Kit structural adaptation (B1287 CS0101 coexistence fix): the kit's 3-state
        // PlotState is nested inside GrayboxSim so it no longer collides at namespace
        // level with FieldCore's 4-state PlotState (Wild/Tilled/Planted/Ripe) in the
        // same runtime assembly. Every kit reference lives inside this class, so the
        // frozen numbers and behavior stay byte-identical.
        public enum PlotState { Wild = 0, Tilled, Planted }
        // ---- Stage1 hook timeline (frozen at E4 authoring - zero loosening) ----
        public const double CaptionT = 0.15;        // hook caption (red line 11, T+0-3s)
        public const double StarterPlotsT = 0.50;  // two starter plots granted
        public const double FirstTillT = 1.00;     // first till (zero-learning-cost 3s hook)
        public const double FirstPlantT = 1.50;    // first plant: tap + sfx + grain pop same tick
        public const double FirstHarvestT = 2.50;  // first harvest
        public const double GoalChainT = 3.00;     // till #3 -&gt; goal 1 + goal 2 double-fire (narrative spine)
        public const double EpisodeUnlockT = 5.50; // episode 1 unlock (serialization teach beat)
        public const double CliffhangerT = 8.00;   // episode 1 complete -&gt; next-episode teaser fires
        // ---- frozen economy tables ----
        public const int PlotCount = 12;
        public const long BaseYield = 100;         // shi per plot per harvest
        public const int CropCount = 3;
        public static readonly string[] CropNames = { "millet", "bean", "wheat" };
        public static readonly string[] PlotTerrainNames =
        {
            "wild_steppe", "sunny_slope", "river_bend", "rocky_ridge", "marsh_hollow", "windy_knoll",
            "old_orchard", "bamboo_edge", "well_side", "south_terrace", "north_field", "cliff_ledge"
        };
        // ---- episode serialization (frozen): per-episode increment targets ----
```

```
// episode gains: e1 3x100=300 / e2 6x100=600 / e3 12x100=1200 /
//   e4 12x100x2(channel)=2400 / e5 12x100x512(festival)=614,400 (slogan promise).
// cumulative ledger after each episode: 300 / 900 / 2100 / 4500 / 618,900.
public const int EpisodeCount = 5;
public static readonly string[] EpisodeNames =
{
    "settling_in", "taking_root", "buying_fields", "digging_the_channel", "harvest_festival"
};
public static readonly long[] EpisodeTargets = { 300, 600, 1200, 2400, 600000 };
public const long Episode3HardTarget = 2640; // doomed-run variant: 2400/2640 = 90.9% -&gt; near-miss band
// ---- guardrail vocabulary bans (U033 / B1227 admission compliance) ----
public static readonly string[] ForbiddenTokens =
{
    "guilty_wife", "criminal_wife", "fa_pei_bian_guan", "zhan_shen_zhang_fu",
    "beheaded", "execution_ground", "torture", "prison_chain"
};
public static readonly string[] PunishTokens = { "punish", "penalty", "confiscate", "land_reset", "dex_wipe", "s
public static readonly string[] ShareFormTokens = { "score", "points", "result", "rank" };
public static int RngCalls = 0;                  // stays 0 forever: everything is deterministic
public const bool AdsRequiredForClosure = false; // ads purely optional
public double SimTime;
public readonly List&lt;SimEvent&gt; Events = new List&lt;SimEvent&gt;();
public readonly PlotState[] Plots = new PlotState[PlotCount];
public readonly int[] PlotTerrain = new int[PlotCount];
public readonly bool[] PremiumPlots = new bool[PlotCount]; // fertility-3 picks, the doubling lineup
public bool ChannelBuilt;
public bool FestivalOn;
public readonly HashSet&lt;int&gt; TerrainDex = new HashSet&lt;int&gt;();   // land dex, never wiped on loss
public readonly HashSet&lt;int&gt; CropDex = new HashSet&lt;int&gt;();     // crop dex, never wiped on loss
public long GrainTotal;                                    // economy ledger single source
public long EpisodeStartGrain;                             // per-episode increment base
public int TillsTotal, PlantsTotal, HarvestsTotal;
public int EpisodesUnlocked;           // count of legal unlocks
public bool[] EpisodeUnlocked = new bool[EpisodeCount];
public bool[] EpisodeCompleted = new bool[EpisodeCount];
public int EpisodeWins, EpisodeLosses, NearMisses;
public int RescueOffers, DoubleOffers, AdsAccepted, ShareCards;
public int RunCount, RunWins, RunLosses;
// narrative goal chain (auto-advancing: completing a goal fires the next one
// on the same tick; first goal must land within 60s - E4 assertion face #3)
public int CurrentGoal;
public double FirstGoalDoneAt = -1.0;
public static readonly string[] GoalTexts =
{
    "till 3 plots of wild land", "bring in the first harvest", "fill the granary to 1000 shi",
    "unlock the next episode", "hold the harvest festival"
};
// ---- Stage3 dense state (perf probe only) ----
public readonly int[] PlotStream = new int[2000];
public readonly double[] FieldHeat = new double[300];
```

```
public readonly double[] GrowthTicker = new double[512];
public readonly double[] GlowField = new double[1024];
public void Log(string type, string detail) { Events.Add(new SimEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
// ==== serialization gate: episode e unlocks only after episode e-1 completed ====
// e is 1-based. E1 is free (the pilot). Every later episode is hard-gated.
// Repeated unlocks and out-of-range indices return false (idempotent gate).
public bool UnlockEpisode(int e)
{
    if (e &lt; 1 || e &gt; EpisodeCount) { Log("episode_unlock_rejected", "out of range"); return false; }
    if (EpisodeUnlocked[e - 1]) { Log("episode_unlock_rejected", "already unlocked"); return false; }
    if (e &gt; 1 &amp;&amp; !EpisodeCompleted[e - 2]) { Log("episode_unlock_rejected", "previous episode not com
    EpisodeUnlocked[e - 1] = true;
    EpisodesUnlocked++;
    Log("episode_unlock", "episode " + e + " unlocked: " + EpisodeNames[e - 1]);
    BeatGoals();
    return true;
}
// ==== run / episode / plot API ====
public void Caption(string text) { Log("caption", text); }
public void StartRun(int runIdx)
{
    RunCount++;
    ChannelBuilt = false;
    FestivalOn = false;
    CurrentGoal = 0;
    FirstGoalDoneAt = -1.0;
    for (int e = 0; e &lt; EpisodeCount; e++) { EpisodeUnlocked[e] = false; EpisodeCompleted[e] = false; }
    int n = PlotTerrainNames.Length;
    for (int i = 0; i &lt; PlotCount; i++)
    {
        Plots[i] = PlotState.Wild;
        PremiumPlots[i] = false;
        PlotTerrain[i] = (((2 * runIdx + i) % n) + n) % n;
        TerrainDex.Add(PlotTerrain[i]);
    }
    Log("run_start", "run " + runIdx + ": exiled to the frontier - twelve plots of border land are surveyed");
}
public void TillPlot(int idx)
{
```

```
    if (idx &lt; 0 || idx &gt;= PlotCount || Plots[idx] != PlotState.Wild) { Log("till_rejected", "not wild land
    Plots[idx] = PlotState.Tilled;
    TillsTotal++;
    Log("till_tap", "till plot " + idx + " (" + PlotTerrainNames[PlotTerrain[idx]] + ")");
    Log("sfx_hoe", "hoe strikes the hard soil");
    BeatGoals();
}
public void PlantPlot(int idx, int cropIdx)
{
    if (idx &lt; 0 || idx &gt;= PlotCount || Plots[idx] != PlotState.Tilled) { Log("plant_rejected", "plot not t
    Plots[idx] = PlotState.Planted;
    PlantsTotal++;
    CropDex.Add(cropIdx % CropCount);
    Log("plant_tap", "plant " + CropNames[cropIdx % CropCount] + " on plot " + idx);
    Log("sfx_seed", "seeds fall into the fresh furrow");
    Log("grain_pop", "granary expectation rises");
    BeatGoals();
}
// episode multiplier: 1 base, x2 once the channel is built, and at the festival
// the premium lineup (2^6 with six picks) and the festival itself (x4) stack -&gt; 512.
public long EpisodeMult()
{
    long m = 1;
    if (ChannelBuilt) m *= 2;
    if (FestivalOn)
    {
        int premium = 0;
        for (int i = 0; i &lt; PlotCount; i++) if (PremiumPlots[i]) premium++;
        m *= (1L &lt;&lt; premium);
        m *= 4;
    }
    return m;
}
public long HarvestPlot(int idx)
{
    if (idx &lt; 0 || idx &gt;= PlotCount || Plots[idx] != PlotState.Planted) { Log("harvest_rejected", "nothing
    long amount = BaseYield * EpisodeMult();
    Plots[idx] = PlotState.Tilled;   // crop rotation: back to tilled, replant next round
    HarvestsTotal++;
    GrainTotal += amount;
    Log("harvest_tap", "harvest plot " + idx + ": +" + amount + " shi");
    BeatGoals();
    return amount;
}
public void BuildChannel()
{
    ChannelBuilt = true;
    Log("channel_build", "the irrigation channel is dug - every harvest doubles");
}
public void PickPremiumPlot(int idx)
```

```
{
    if (idx &lt; 0 || idx &gt;= PlotCount) return;
    PremiumPlots[idx] = true;
    Log("premium_pick", "plot " + idx + " is coaxed to top fertility - it joins the doubling lineup");
}
public void StartFestival()
{
    FestivalOn = true;
    Log("festival_on", "the harvest festival begins - the whole estate answers");
    BeatGoals();
}
public void StartEpisode(int e)
{
    EpisodeStartGrain = GrainTotal;
    Log("episode_start", "episode " + e + " begins: " + EpisodeNames[e - 1]);
}
public long EpisodeGain() { return GrainTotal - EpisodeStartGrain; }
public bool ResolveEpisode(int e, long target)
{
    long gain = EpisodeGain();
    if (gain &gt;= target)
    {
        EpisodeWins++;
        EpisodeCompleted[e - 1] = true;
        Log("episode_won", "episode " + e + " clears the target");
        Cliffhanger(e);
        DoubleOffer();
        return true;
    }
    EpisodeLosses++;
    if ((double)gain &gt;= target * 0.9 - 1e-9)
    {
        NearMisses++;
        Log("near_miss", "so close - the episode fell just short, rescue offers pop up");
        RescueOffer();
    }
    else
    {
        Log("episode_lost_clean", "the episode falls short, the run ends here");
    }
    return false;
}
// Cliffhanger: the binge hook. Fires on every episode win (the next-episode
// teaser). The drama-serialization increment, asserted as its own beat.
public void Cliffhanger(int e)
{
    if (e &lt; EpisodeCount)
        Log("cliffhanger_fire", "next-episode teaser: the county magistrate rides in - her smile hides a blade")
    else
        Log("cliffhanger_fire", "season finale teaser: the frontier estate stands - next season, the capital cal
```

```
}
// Near-miss rescue (C013 point density): three optional rewarded-video outs.
// The bot never accepts; the loop still closes (zero forced ads).
public void RescueOffer()
{
    Log("rescue_offer", "optional video: revive and retry this episode?");
    Log("rescue_offer", "optional video: replant the failed plots?");
    Log("rescue_offer", "optional video: double the granary to pass?");
    RescueOffers += 3;
}
public void DoubleOffer()
{
    DoubleOffers++;
    Log("double_offer", "optional video: double this episode harvest?");
}
public void RunWon() { RunWins++; Log("run_won", "five episodes cleared - the wasteland is an estate now"); }
public void RunLost(int runIdx)
{
    RunLosses++;
    Log("run_lost", "run " + runIdx + " ends, the land dex keeps everything");
}
// Share face (B904 guardrail #2 analog): episode achievement / land-dex card
// with an X032-style code. Score-form display stays out of the payload.
public void ShareCard()
{
    ShareCards++;
    StringBuilder sb = new StringBuilder("EF1-");
    for (int i = 0; i &lt; PlotTerrainNames.Length; i++)
        if (TerrainDex.Contains(i)) sb.Append(i.ToString("X", CultureInfo.InvariantCulture));
    Log("share_card", "episode achievement card code=" + sb + " land_dex=" + TerrainDex.Count + "/" + PlotTerrai
}
// ==== narrative goal chain: auto-advance, same tick ====
void BeatGoals()
{
    bool advanced = true;
    while (advanced &amp;&amp; CurrentGoal &lt; GoalTexts.Length)
    {
        advanced = false;
        switch (CurrentGoal)
        {
            case 0:
                if (TillsTotal &gt;= 3) { GoalDone(); advanced = true; }
                break;
            case 1:
                if (HarvestsTotal &gt;= 1) { GoalDone(); advanced = true; }
                break;
            case 2:
                if (GrainTotal &gt;= 1000) { GoalDone(); advanced = true; }
                break;
            case 3:
```

```
                if (EpisodesUnlocked &gt;= 2) { GoalDone(); advanced = true; }
                break;
            case 4:
                if (FestivalOn) { GoalDone(); advanced = true; }
                break;
        }
    }
}
void GoalDone()
{
    if (FirstGoalDoneAt &lt; 0 &amp;&amp; CurrentGoal == 0) FirstGoalDoneAt = SimTime;
    Log("goal_complete", "goal done: " + GoalTexts[CurrentGoal]);
    CurrentGoal++;
    if (CurrentGoal &lt; GoalTexts.Length)
        Log("goal_fire", "next goal: " + GoalTexts[CurrentGoal]);
}
// ==== Stage3 dense estate (perf probe only) ====
public void BuildDense()
{
    for (int i = 0; i &lt; 2000; i++) PlotStream[i] = (i * 7) % 12;
    for (int t = 0; t &lt; 300; t++) FieldHeat[t] = ((t * 13) % 17) * 0.05;
    for (int m = 0; m &lt; 512; m++) GrowthTicker[m] = ((m * 11) % 19) * 0.05;
    for (int g = 0; g &lt; 1024; g++) GlowField[g] = ((g * 5) % 23) * 0.05;
}
// Dense festival resolve: counting pass over the 2000-plot stream, window
// enumeration for the best contiguous-plot yield, then heat / ticker / glow
// smoothing passes.
public bool DenseFestivalResolve(int idx)
{
    int[] cnt = new int[12];
    for (int i = 0; i &lt; 2000; i++) cnt[PlotStream[i] % 12]++;
    long denseMult = 1L &lt;&lt; (6 + (idx % 7));
    long best = 0;
    for (int p = 0; p &lt; 12; p++)
    {
        if (cnt[p] &gt;= 1) { long s = BaseYield * 1 * denseMult; if (s &gt; best) best = s; }
        if (cnt[p] &gt;= 2) { long s = BaseYield * 2 * denseMult; if (s &gt; best) best = s; }
    }
    for (int len = 3; len &lt;= 12; len++)
    {
        for (int start = 0; start + len - 1 &lt; 12; start++)
        {
            bool ok = true;
            for (int p = start; p &lt; start + len; p++) if (cnt[p] &lt; 1) { ok = false; break; }
            if (ok) { long s = BaseYield * len * denseMult; if (s &gt; best) best = s; }
        }
    }
    for (int t = 0; t &lt; 300; t++) FieldHeat[t] = (FieldHeat[t] + (((t * 13) % 17) * 0.05)) * 0.5;
    for (int m = 0; m &lt; 512; m++) GrowthTicker[m] = (GrowthTicker[m] + GrowthTicker[(m * 31 + 7) % 512]) * 0.
    for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
```

```
            return best &gt; 0;
        }
        // Raw idle tick: stream drift + ticker relax + glow relax.
        public void DenseTick()
        {
            for (int i = 0; i &lt; 2000; i++) PlotStream[i] = (PlotStream[i] + 1) % 12;
            for (int m = 0; m &lt; 512; m++) GrowthTicker[m] = (GrowthTicker[m] + GrowthTicker[(m * 31 + 7) % 512]) * 0.
            for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
        }
    }
}
// LocaleService.cs
// LocaleService.cs - S1 T010 c2 caption-key localization core for ExileFarm (B1284).
// The c1 pages host rendered the frozen English mirrors; this core externalizes the
// copy by CaptionKey: the host resolves every player-visible string through
// ByKey(CaptionKey) against the shipped cfg_i18n table (Resources/cfg_i18n.json,
// the ConfigValidator closed set - the validator mirror and this census stay in
// sync under the X1004 twin law). Pure C#: the host parses the TextAsset with
// JsonUtility and injects the rows; the strict faces mirror the kit ctor law
// (duplicate keys reject, unknown keys reject) so a missing key fails loud at
// gate time, never silently at play time. The {N} slot face serves the slogan /
// share card rows ("reached episode {N}"). Default language is zh (the
// Chinese-facing vertical); SetLang switches the ByKey face for the en mirror
// path (the frozen mirrors in PagesGate pin byte-equal to the en rows - the
// AudioCheck twin face).
using System;
using System.Collections.Generic;
namespace Sy.ExileFarm
{
    public sealed class LocaleService
    {
        public const string DefaultLang = "zh";
        public const string EnLang = "en";
        readonly Dictionary&lt;string, string&gt; _en = new Dictionary&lt;string, string&gt;();
        readonly Dictionary&lt;string, string&gt; _zh = new Dictionary&lt;string, string&gt;();
        public string Lang = DefaultLang;
        public LocaleService(IList&lt;string&gt; keys, IList&lt;string&gt; en, IList&lt;string&gt; zh)
        {
            if (keys == null || en == null || zh == null)
                throw new ArgumentException("locale table rows must not be null");
            if (keys.Count != en.Count || keys.Count != zh.Count)
                throw new ArgumentException("locale table must carry en/zh for every key");
            for (int i = 0; i &lt; keys.Count; i++)
            {
                if (keys[i] == null || keys[i].Length == 0)
                    throw new ArgumentException("locale key " + i + " must be a CaptionKey");
                if (_en.ContainsKey(keys[i]))
                    throw new ArgumentException("duplicate locale key: " + keys[i]);
                if (en[i] == null || en[i].Length == 0 || zh[i] == null || zh[i].Length == 0)
                    throw new ArgumentException("locale key " + keys[i] + " must carry both en and zh copy");
```

```
                _en[keys[i]] = en[i];
                _zh[keys[i]] = zh[i];
            }
        }
        public int KeyCount { get { return _en.Count; } }
        public bool HasKey(string key) { return _en.ContainsKey(key); }
        /// &lt;summary&gt;Resolve copy by CaptionKey in the current language. Strict:
        /// an unknown key throws (fail loud at gate time - the closed-set census
        /// gate guarantees every referenced key exists).&lt;/summary&gt;
        public string ByKey(string key)
        {
            return ByKey(Lang, key);
        }
        /// &lt;summary&gt;Resolve copy by CaptionKey in an explicit language.&lt;/summary&gt;
        public string ByKey(string lang, string key)
        {
            if (key == null) throw new ArgumentException("CaptionKey must not be null");
            Dictionary&lt;string, string&gt; table = lang == EnLang ? _en : _zh;
            string value;
            if (!table.TryGetValue(key, out value))
                throw new ArgumentException("unknown CaptionKey: " + key);
            return value;
        }
        /// &lt;summary&gt;{N} slot face: replace the literal slot token with the value
        /// ("reached episode {N}" -&gt; "reached episode 1"). Returns the input
        /// untouched when the slot is absent.&lt;/summary&gt;
        public static string ApplySlot(string text, string slot, string value)
        {
            if (text == null || slot == null) return text;
            return text.Replace("{" + slot + "}", value);
        }
    }
}
// PagesGate.cs
// PagesGate.cs - S1 T010 pages gate for ExileFarm (B1279, G23 CardRogue PagesGate
// pattern ported onto the G26 frozen faces).
// Three acceptance faces from the taskbook row T010 (c1 slice): the frozen page
// list (S99_Boot / S01_Homestead / S02_Codex / S03_Share - zero missing, zero
// extra, every canvas carries a U184 scaler + raycaster, built by the view host in
// PagesHost.cs), the six-beat teach table (demand-doc section 3, frozen at E4
// authoring: 0.15 hook caption / 1.00 first till / 1.50 first-feedback triple /
// 2.50 first narrative goal done / 5.50 episode 1 unlock / 8.00 cliffhanger teaser),
// and the save face (grain / estate stage+ledger / terrain dex / crop dex /
// episodes completed / ad grant stamps - strict compose-parse round-trip through
// ISaveService; the AdCore ExportLedger save.adcaps data face wired here per the
// T009 contract). The teach drive replays the canonical S1 opening on the real
// cores: first till 1.00, plant triple 1.50, till #2 2.00 (kit Stage1 pacing), then
// at 2.50 till #3 + first harvest (1.50 + growth 1.00) land on the same tick so the
// goal chain double-fires goal 1 + goal 2 (the E4 spine), episode 1 unlocks at
// 5.50 (pilot free), three rows ripen together at 7.00 (+300 shi = the E1 target),
```

```
// and at 8.00 episode 1 clears the 300 target - win, cliffhanger teaser, one
// optional double offer. Pure C# (no UnityEngine) so Editor gates drive it
// headless and the view host consumes the same frozen faces. Every number is a
// frozen anchor (E4 kit / demand-doc section 3 / shipped cores) - zero re-tuning,
// zero RNG.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using Sy.Core;
namespace Sy.ExileFarm
{
    /// &lt;summary&gt;
    /// Frozen page registry (demand-doc section 3 page list) and teach anchors.
    /// Zero re-tuning: S2+ widens flavor, never the page set, the cell counts or
    /// the beat times.
    /// &lt;/summary&gt;
    public static class PageFrozen
    {
        public static readonly string[] Names = { "S99_Boot", "S01_Homestead", "S02_Codex", "S03_Share" };
        public const int PlotCells = 12;   // homestead wild-plot grid (FieldCore.PlotCount face)
        public const int CropSlots = 3;     // three-crop roster strip (FieldCore.CropCount face)
        public const int CodexCells = 12;   // twelve-grid dex wall (CodexCore.FrozenTerrainCount face)
        public const string SaveSlot = "exilefarm_save";
        // section-3 beats no shipped core carries as a constant:
        public const double CaptionT = 0.15;         // hook caption (kit GrayboxSim.CaptionT verbatim)
        public const double TeachSecondTillT = 2.00;  // till #2 (kit Stage1 pacing, 2.00s)
        // U184 per-canvas law (the view host consumes these):
        public const int RefWidth = 1080;
        public const int RefHeight = 1920;
        public static bool IsPage(string name)
        {
            for (int i = 0; i &lt; Names.Length; i++) if (Names[i] == name) return true;
            return false;
        }
    }
    /// &lt;summary&gt;One red-line-11 teach beat: frozen time + id + page + i18n caption
    /// key + the frozen English mirror of the key (the c1 view face; the c2 slice
    /// resolves captions through LocaleService by CaptionKey) + the event anchor the
    /// beat teaches. Pure data, zero logic.&lt;/summary&gt;
    public sealed class TeachBeat
    {
        public readonly double T;
        public readonly string Id;
        public readonly string Page;
        public readonly string CaptionKey;
        public readonly string CaptionMirror;
        public readonly string Anchor;
        public TeachBeat(double t, string id, string page, string captionKey, string captionMirror, string anchor)
        {
```

```
        T = t; Id = id; Page = page; CaptionKey = captionKey; CaptionMirror = captionMirror; Anchor = anchor;
    }
}
public static class PagesGate
{
    // ---- the frozen six-beat table (demand-doc section 3, E4 authoring). Five
    // beats bind byte-equal onto the shipped core constants; the caption beat
    // binds onto the PageFrozen kit-verbatim constant. Monotonic by construction.
    public static readonly TeachBeat[] TeachBeats =
    {
        new TeachBeat(PageFrozen.CaptionT,            "hook",           "S01_Homestead", "caption_hook",        "Exi
        new TeachBeat(FieldCore.FirstTillT,          "first_till",     "S01_Homestead", "teach_first_till",    "Firs
        new TeachBeat(FieldCore.FirstPlantT,         "first_triple",   "S01_Homestead", "teach_first_harvest", "Firs
        new TeachBeat(GoalCore.FirstGoalBeatT,       "first_goal",     "S01_Homestead", "teach_first_goal",     "Fir
        new TeachBeat(EpisodeCore.EpisodeUnlockT,    "episode_unlock", "S01_Homestead", "teach_episode_unlock","Epis
        new TeachBeat(EpisodeCore.CliffhangerT,      "cliffhanger",    "S01_Homestead", "teach_cliffhanger",    "End
    };
    // ---- the canonical S1 teach opening, replayed on the real cores with the
    // frozen beat times (the view paces the same beats in wall time; the gate
    // replays them in logical time). 1.00: first till. 1.50: the plant triple
    // (tap + seed sfx + grain pop). 2.00: till #2. 2.50: till #3 + GrowTo +
    // first harvest (1.50 + growth 1.00) on the same tick -&gt; BeatGoals cascades
    // goal 1 (three tills) + goal 2 (first harvest) - the E4 double-fire spine.
    // 5.50: episode 1 unlocks (the pilot is free). 6.00: episode 1 starts and
    // three rows go in; 7.00: they ripen together and three harvests land
    // (+300 shi = the E1 target, cumulative ledger face 300). 8.00: episode 1
    // clears the target - win, cliffhanger teaser, one optional double offer.
    public static void RunTeachDrive(FieldCore field, GoalCore goal, EpisodeCore episode)
    {
        if (field == null || goal == null || episode == null) return;
        field.SimTime = PageFrozen.CaptionT;            // 0.15 hook caption
        field.Log("caption", TeachBeats[0].CaptionMirror);
        field.SimTime = FieldCore.FirstTillT;           // 1.00 first till
        field.TillPlot(0);
        field.SimTime = FieldCore.FirstPlantT;          // 1.50 first feedback triple
        field.PlantPlot(0, 0);
        field.SimTime = PageFrozen.TeachSecondTillT;    // 2.00 till #2
        field.TillPlot(1);
        field.SimTime = GoalCore.FirstGoalBeatT;         // 2.50 till #3 + first harvest
        field.TillPlot(2);
        field.GrowTo(GoalCore.FirstGoalBeatT);
        field.HarvestPlot(0);
        goal.SimTime = GoalCore.FirstGoalBeatT;
        goal.BeatGoals(field, episode);                 // goal 1 + goal 2 same tick
        episode.SimTime = EpisodeCore.EpisodeUnlockT;   // 5.50 episode 1 unlock
        episode.UnlockEpisode(1);
        goal.SimTime = EpisodeCore.EpisodeUnlockT;
        goal.BeatGoals(field, episode);
        episode.SimTime = 6.00;                         // 6.00 episode 1 starts
        episode.StartEpisode(1);
```

```
        field.SimTime = 6.00;
        field.PlantPlot(0, 0);
        field.PlantPlot(1, 1);
        field.PlantPlot(2, 2);
        field.GrowTo(7.00);                          // 7.00 three rows ripen together
        long g0 = field.HarvestPlot(0);
        long g1 = field.HarvestPlot(1);
        long g2 = field.HarvestPlot(2);
        episode.SimTime = 7.00;                      // presenter wiring: harvest income feeds the episode ledger
        episode.CreditGrain(g0);
        episode.CreditGrain(g1);
        episode.CreditGrain(g2);
        goal.SimTime = 7.00;
        goal.BeatGoals(field, episode);
        episode.SimTime = EpisodeCore.CliffhangerT;  // 8.00 E1 clears the 300 target
        episode.ResolveEpisode(1, episode.EpisodeTargets[0]);
        goal.SimTime = EpisodeCore.CliffhangerT;
        goal.BeatGoals(field, episode);
    }
    // ---- save face: grain / estate / dex / episodes / adcaps -----------------
    // Service round-trip helpers: the runtime asmdef carries the Sy.Core service
    // face (the editor asmdef does not reference it), so the editor gate drives
    // the ISaveService round-trip through these two helpers.
    public static bool SaveFaceThroughService(SaveFace face)
    {
        ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, SaveCodec.Compose(face));
        return true;
    }
    public static string LoadFacePayload()
    {
        return ServiceLocator.Get&lt;ISaveService&gt;().Load(PageFrozen.SaveSlot);
    }
    /// &lt;summary&gt;The persisted face: the field grain ledger, the estate stage +
    /// cumulative ledger, the collected dex (terrain + crop sets, never wiped on
    /// loss - zero-punishment law), the completed episodes (1-based, the unlock
    /// gate derives the rest) and the ad grant stamps (the T009 AdCore
    /// ExportLedger save.adcaps data face, wall-clock seconds).&lt;/summary&gt;
    public sealed class SaveFace
    {
        public long Grain;
        public int EstateStage;
        public long EstateLedger;
        public readonly List&lt;int&gt; TerrainDex = new List&lt;int&gt;();
        public readonly List&lt;int&gt; CropDex = new List&lt;int&gt;();
        public readonly List&lt;int&gt; EpisodesCompleted = new List&lt;int&gt;();
        public readonly List&lt;long&gt; AdGrantStamps = new List&lt;long&gt;();
    }
    /// &lt;summary&gt;
    /// Strict save codec (ShareCodec twin law): fixed prefix, fixed section
    /// order, strict token grammar, roster-bounded indices, and a canonical
```

```
/// compose so Compose(Parse(Compose(x))) == Compose(x) always holds.
/// Format (pure ASCII, zero hard caps vocabulary):
///   EFSAVE1|G=400|E=1,4500|T=0,1,2|C=0,1|D=1|A=3600,7200
/// The dex sections compose sorted + deduped (set semantics); the episodes
/// compose sorted + deduped (1-based); the grant stamps keep ledger order.
/// &lt;/summary&gt;
public static class SaveCodec
{
    public const string Prefix = "EFSAVE1";
    const int StageMax = 5; // EstateCore.EstateStageCount
    public static string Compose(SaveFace face)
    {
        StringBuilder sb = new StringBuilder();
        sb.Append(Prefix);
        sb.Append("|G=").Append(face.Grain.ToString(CultureInfo.InvariantCulture));
        sb.Append("|E=").Append(face.EstateStage.ToString(CultureInfo.InvariantCulture))
          .Append(',').Append(face.EstateLedger.ToString(CultureInfo.InvariantCulture));
        sb.Append("|T=");
        AppendInts(sb, SortedUnique(face.TerrainDex));
        sb.Append("|C=");
        AppendInts(sb, SortedUnique(face.CropDex));
        sb.Append("|D=");
        AppendInts(sb, SortedUnique(face.EpisodesCompleted));
        sb.Append("|A=");
        for (int i = 0; i &lt; face.AdGrantStamps.Count; i++)
        {
            if (i &gt; 0) sb.Append(',');
            sb.Append(face.AdGrantStamps[i].ToString(CultureInfo.InvariantCulture));
        }
        return sb.ToString();
    }
    static List&lt;int&gt; SortedUnique(List&lt;int&gt; src)
    {
        List&lt;int&gt; list = new List&lt;int&gt;(src);
        list.Sort();
        for (int i = 1; i &lt; list.Count; i++) if (list[i] == list[i - 1]) { list.RemoveAt(i); i--; }
        return list;
    }
    static void AppendInts(StringBuilder sb, List&lt;int&gt; list)
    {
        for (int i = 0; i &lt; list.Count; i++)
        {
            if (i &gt; 0) sb.Append(',');
            sb.Append(list[i].ToString(CultureInfo.InvariantCulture));
        }
    }
    public static bool TryParse(string payload, out SaveFace face)
    {
        face = null;
        if (payload == null) return false;
```

```
            string[] sections = payload.Split('|');
            if (sections.Length != 7 || sections[0] != Prefix) return false;
            if (!sections[1].StartsWith("G=", StringComparison.Ordinal) ||
                !sections[2].StartsWith("E=", StringComparison.Ordinal) ||
                !sections[3].StartsWith("T=", StringComparison.Ordinal) ||
                !sections[4].StartsWith("C=", StringComparison.Ordinal) ||
                !sections[5].StartsWith("D=", StringComparison.Ordinal) ||
                !sections[6].StartsWith("A=", StringComparison.Ordinal)) return false;
            SaveFace f = new SaveFace();
            if (!long.TryParse(sections[1].Substring(2), NumberStyles.None, CultureInfo.InvariantCulture, out f.Grai
                f.Grain &lt; 0) return false;
            string e = sections[2].Substring(2);
            int comma = e.IndexOf(',');
            if (comma &lt;= 0) return false;
            if (!int.TryParse(e.Substring(0, comma), NumberStyles.None, CultureInfo.InvariantCulture, out f.EstateSt
                f.EstateStage &lt; 0 || f.EstateStage &gt; StageMax) return false;
            if (!long.TryParse(e.Substring(comma + 1), NumberStyles.None, CultureInfo.InvariantCulture, out f.Estate
                f.EstateLedger &lt; 0) return false;
            if (!ParseIntList(sections[3].Substring(2), f.TerrainDex, 0, FieldCore.PlotCount - 1)) return false;
            if (!ParseIntList(sections[4].Substring(2), f.CropDex, 0, FieldCore.CropCount - 1)) return false;
            if (!ParseIntList(sections[5].Substring(2), f.EpisodesCompleted, 1, EpisodeCore.EpisodeCount)) return fa
            string a = sections[6].Substring(2);
            if (a.Length &gt; 0)
            {
                string[] stamps = a.Split(',');
                long ts;
                for (int i = 0; i &lt; stamps.Length; i++)
                {
                    if (!long.TryParse(stamps[i], NumberStyles.None, CultureInfo.InvariantCulture, out ts) ||
                        ts &lt; 0) return false;
                    f.AdGrantStamps.Add(ts);
                }
            }
            face = f;
            return true;
        }
        static bool ParseIntList(string s, List&lt;int&gt; into, int min, int max)
        {
            if (s.Length == 0) return true;
            string[] parts = s.Split(',');
            int v;
            for (int i = 0; i &lt; parts.Length; i++)
            {
                if (!int.TryParse(parts[i], NumberStyles.None, CultureInfo.InvariantCulture, out v)) return false;
                if (v &lt; min || v &gt; max) return false;   // roster-bounded
                into.Add(v);
            }
            return true;
        }
    }
```

```
    }
}
// PagesHost.cs
// PagesHost.cs - S1 T010 procedural UGUI pages host for ExileFarm (B1279, G23
// CardRogue PagesHost pattern ported onto the G26 frozen pages, X027 paradigm:
// zero prefabs, zero scene serialization - the main scene only carries the camera,
// the shell canvas and the EventSystem; every page is built in code at runtime).
// Four frozen pages (PageFrozen registry): S99_Boot splash, S01_Homestead (field bg
// + twelve wild-plot cells + three crop slots + the estate panel + the episode
// progress bar + the HUD entries codex/share/offline + the four action buttons +
// the cliffhanger teaser banner), S02_Codex twelve-grid wall, S03_Share (share
// frame + EF1- code label + the episode card face).
// U184 per-canvas law: every page canvas carries a CanvasScaler locked to
// ScaleWithScreenSize at the frozen 1080x1920 portrait reference plus a
// GraphicRaycaster. The ProjectSetup placeholder canvas (the "S26_Main Main"
// label shell) is absorbed on build - placeholder shell law, zero stray canvas.
// Teach pacing walks the frozen PagesGate six-beat table in wall time; captions
// resolve through LocaleService by CaptionKey (T010 c2 externalization, zh
// default) and every beat drives the same canonical opening
// on the real cores (FieldCore / GoalCore / EpisodeCore - the E4 Stage1 timeline).
// Save wiring mirrors the field grain ledger + the completed episodes into the
// SaveFace and lands it through ISaveService (the save.adcaps data face contracted
// in T009 AdCore).
using System;
using System.Collections;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.ExileFarm
{
    public sealed class PagesHost : MonoBehaviour
    {
        static Font _font;
        static LocaleService _locale;
        readonly Dictionary&lt;string, GameObject&gt; _pages = new Dictionary&lt;string, GameObject&gt;();
        FieldCore _field;
        GoalCore _goal;
        EpisodeCore _episode;
        public static Font SharedFont
        {
            get
            {
                if (_font == null) _font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
                return _font;
            }
        }
        [Serializable] class I18nRow { public string key; public string en; public string zh; }
        [Serializable] class I18nTable { public List&lt;I18nRow&gt; strings; }
```

```
/// &lt;summary&gt;Locale face (T010 c2): parse the shipped cfg_i18n table (the
/// ConfigValidator closed set) once, serve ByKey(CaptionKey) lookups - zh
/// default, the Chinese-facing vertical. The strict LocaleService ctor
/// mirrors the kit law: a broken table fails loud here, never silently at
/// play time.&lt;/summary&gt;
public static LocaleService Locale
{
    get
    {
        if (_locale == null)
        {
            TextAsset asset = Resources.Load&lt;TextAsset&gt;("cfg_i18n");
            if (asset == null) throw new ArgumentException("cfg_i18n table missing from Resources");
            I18nTable table = JsonUtility.FromJson&lt;I18nTable&gt;(asset.text);
            if (table == null || table.strings == null || table.strings.Count == 0)
                throw new ArgumentException("cfg_i18n table is empty");
            List&lt;string&gt; keys = new List&lt;string&gt;(table.strings.Count);
            List&lt;string&gt; en = new List&lt;string&gt;(table.strings.Count);
            List&lt;string&gt; zh = new List&lt;string&gt;(table.strings.Count);
            for (int i = 0; i &lt; table.strings.Count; i++)
            {
                keys.Add(table.strings[i].key);
                en.Add(table.strings[i].en);
                zh.Add(table.strings[i].zh);
            }
            _locale = new LocaleService(keys, en, zh);
        }
        return _locale;
    }
}
// wireframe palette (warm frontier neutrals; the gold is the section-4
// accent candidate #E8A33D as a display face only - the accent itself stays
// unfrozen until the S0 DiffGate batch, this is not the frozen accent claim)
static readonly Color32 Soil = new Color32(0x6B, 0x4A, 0x2F, 0xFF);
static readonly Color32 SoilDark = new Color32(0x4A, 0x35, 0x22, 0xFF);
static readonly Color32 Paper = new Color32(0xF5, 0xEC, 0xD8, 0xFF);
static readonly Color32 Ink = new Color32(0x2B, 0x1D, 0x10, 0xFF);
static readonly Color32 PlotCell = new Color32(0x8A, 0x64, 0x3E, 0xFF);
static readonly Color32 Seedling = new Color32(0x6E, 0x8B, 0x3D, 0xFF);
static readonly Color32 Gold = new Color32(0xE8, 0xA3, 0x3D, 0xFF); // accent candidate display face
/// &lt;summary&gt;Boot wiring face: find-or-create the host in the freshly loaded
/// main scene (called by BootRunner after the scene switch; idempotent).&lt;/summary&gt;
public static PagesHost EnsureInScene()
{
    PagesHost existing = FindObjectOfType&lt;PagesHost&gt;();
    if (existing != null) return existing;
    GameObject go = new GameObject("PagesHost");
    return go.AddComponent&lt;PagesHost&gt;();
}
void Awake()
```

```
{
    BuildAll();
}
IEnumerator Start()
{
    // splash beats: hold the boot page, then hand the homestead the teach loop
    yield return new WaitForSeconds(0.8f);
    if (_pages.ContainsKey(PageFrozen.Names[0])) _pages[PageFrozen.Names[0]].SetActive(false);
    if (_pages.ContainsKey(PageFrozen.Names[1])) _pages[PageFrozen.Names[1]].SetActive(true);
    yield return TeachLoop();
}
// ==== build face =====================================================
public void BuildAll()
{
    AbsorbPlaceholder();
    EnsureEventSystem();
    for (int i = 0; i &lt; PageFrozen.Names.Length; i++)
    {
        string name = PageFrozen.Names[i];
        if (_pages.ContainsKey(name)) continue;
        _pages[name] = BuildPage(name);
    }
    BuildBootContent();
    BuildHomesteadContent();
    BuildCodexContent();
    BuildShareContent();
    _pages[PageFrozen.Names[0]].SetActive(true);   // splash first
    _pages[PageFrozen.Names[1]].SetActive(true);   // homestead under the splash
    _pages[PageFrozen.Names[2]].SetActive(false);  // codex overlay
    _pages[PageFrozen.Names[3]].SetActive(false);  // share overlay
}
/// &lt;summary&gt;Placeholder shell law: the ProjectSetup-generated root "Canvas"
/// (with its "S26_Main Main" label) is a skeleton face - absorb it so the
/// built pages are the only canvases (U184: zero stray canvas without scaler).&lt;/summary&gt;
void AbsorbPlaceholder()
{
    Canvas[] roots = FindObjectsOfType&lt;Canvas&gt;();
    for (int i = 0; i &lt; roots.Length; i++)
    {
        Canvas c = roots[i];
        if (c.transform.parent != null) continue;
        if (PageFrozen.IsPage(c.name)) continue;
        if (c.name == "Canvas")
        {
            if (Application.isPlaying) Destroy(c.gameObject);
            else DestroyImmediate(c.gameObject);
        }
    }
}
void EnsureEventSystem()
```

```
{
    if (FindObjectOfType&lt;EventSystem&gt;() != null) return;
    GameObject esGo = new GameObject("EventSystem");
    esGo.AddComponent&lt;EventSystem&gt;();
    esGo.AddComponent&lt;StandaloneInputModule&gt;();
}
GameObject BuildPage(string name)
{
    GameObject go = new GameObject(name);
    Canvas canvas = go.AddComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    // U184 per-canvas law: scaler locked to ScaleWithScreenSize at the frozen
    // portrait reference + raycaster on every page canvas.
    CanvasScaler scaler = go.AddComponent&lt;CanvasScaler&gt;();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(PageFrozen.RefWidth, PageFrozen.RefHeight);
    scaler.matchWidthOrHeight = 0.5f;
    go.AddComponent&lt;GraphicRaycaster&gt;();
    return go;
}
// ---- shared widget helpers ----------------------------------------
static RectTransform NewRect(Transform parent, string name)
{
    GameObject go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    return go.GetComponent&lt;RectTransform&gt;();
}
static Image NewImage(Transform parent, string name, Color32 color, float w, float h, float x, float y)
{
    RectTransform rt = NewRect(parent, name);
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = new Vector2(x, y);
    Image img = rt.gameObject.AddComponent&lt;Image&gt;();
    img.color = color;
    return img;
}
static Text NewText(Transform parent, string name, string str, int size, float w, float h, float x, float y, Col
{
    RectTransform rt = NewRect(parent, name);
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = new Vector2(x, y);
    Text t = rt.gameObject.AddComponent&lt;Text&gt;();
    t.font = SharedFont;
    t.text = str;
    t.fontSize = size;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = color;
    return t;
}
// ---- S99_Boot: splash with the title + slogan -----------------------
```

```
// c2 face: the labels resolve through LocaleService by CaptionKey (zh
// default; the {N} slot lands as episode 1 at boot).
void BuildBootContent()
{
    Transform page = _pages["S99_Boot"].transform;
    NewImage(page, "SplashBg", SoilDark, 0, 0, 0, 0).rectTransform.StretchFull();
    NewText(page, "PageTitleLabel", Locale.ByKey("ui_title"), 96, 900, 160, 0, 260, Paper);
    NewText(page, "PageSloganLabel", LocaleService.ApplySlot(Locale.ByKey("ui_slogan"), "N", "1"), 40, 1000, 80,
}
// ---- S01_Homestead: the frontier homestead ----------------------------
void BuildHomesteadContent()
{
    Transform page = _pages["S01_Homestead"].transform;
    NewImage(page, "HomesteadBg", Soil, 0, 0, 0, 0).rectTransform.StretchFull();
    // estate panel (the five-stage base face) + episode progress bar
    NewImage(page, "EstatePanel", SoilDark, 1000, 150, 0, 800);
    NewImage(page, "EpisodeProgressBar", PlotCell, 1000, 36, 0, 660);
    // twelve wild-plot cells (section-3 grid face)
    for (int i = 0; i &lt; PageFrozen.PlotCells; i++)
    {
        int col = i % 4, row = i / 4;
        NewImage(page, "PlotCell_" + i, PlotCell, 230, 230, -360 + col * 240, 380 - row * 270);
    }
    // three-crop roster strip
    for (int i = 0; i &lt; PageFrozen.CropSlots; i++)
        NewImage(page, "CropSlot_" + i, Seedling, 160, 160, -260 + i * 260, -420);
    // four action buttons (till / plant / harvest / upgrade - section-3 taps)
    NewImage(page, "TillButton", Paper, 240, 90, -390, -620);
    NewImage(page, "PlantButton", Paper, 240, 90, -130, -620);
    NewImage(page, "HarvestButton", Paper, 240, 90, 130, -620);
    NewImage(page, "UpgradeButton", Paper, 240, 90, 390, -620);
    // cliffhanger teaser banner (the 8.00 binge-hook seat)
    NewImage(page, "TeaserBanner", Ink, 1000, 110, 0, -730);
    // c2 face: the teach caption label exists from the build face and
    // stays EMPTY until the runtime fills it by CaptionKey (zero
    // hardcoded copy at build; T010 c2 A4).
    NewText(page, "TeachCaptionLabel", "", 40, 1000, 70, 0, -300, Paper);
    // HUD entries: codex / share / offline-summary (section-3 HUD face)
    NewImage(page, "HudCodexEntry", Paper, 300, 90, -330, -880);
    NewImage(page, "HudShareEntry", Paper, 300, 90, 0, -880);
    NewImage(page, "HudOfflineEntry", Paper, 300, 90, 330, -880);
}
// ---- S02_Codex: the twelve-grid dex wall ------------------------------
void BuildCodexContent()
{
    Transform page = _pages["S02_Codex"].transform;
    NewImage(page, "CodexBg", SoilDark, 0, 0, 0, 0).rectTransform.StretchFull();
    NewText(page, "CodexTitleLabel", Locale.ByKey("ui_codex"), 64, 900, 120, 0, -780, Paper);
    for (int i = 0; i &lt; PageFrozen.CodexCells; i++)
    {
```

```
        int col = i % 3, row = i / 3;
        NewImage(page, "CodexCell_" + i, PlotCell, 280, 280, -320 + col * 320, 520 - row * 320);
    }
}
// ---- S03_Share: the episode card ----------------------------------
void BuildShareContent()
{
    Transform page = _pages["S03_Share"].transform;
    NewImage(page, "ShareDim", Ink, 0, 0, 0, 0).rectTransform.StretchFull();
    NewImage(page, "ShareFrame", Paper, 700, 1200, 0, 0);
    Transform frame = page.Find("ShareFrame");
    NewText(frame, "ShareCodeLabel", ShareCodecCore.CodePrefix, 44, 620, 90, 0, 480, Ink);
    NewImage(frame, "ShareEpisodeCard", Soil, 620, 620, 0, 40);
}
// ==== runtime faces ================================================
/// &lt;summary&gt;Teach pacing: walk the frozen six-beat table in wall time, show
/// the caption mirrors on the homestead caption label, and run the same
/// canonical opening on the real cores (the sequence PagesGate.RunTeachDrive
/// replays in logical time); the save face lands at the end.&lt;/summary&gt;
IEnumerator TeachLoop()
{
    double[] growth = { FieldCore.FrozenGrowthSeconds, FieldCore.FrozenGrowthSeconds, FieldCore.FrozenGrowthSeco
    _field = new FieldCore(FieldCore.CropNames, growth, FieldCore.PlotTerrainNames);
    _goal = new GoalCore(GoalCore.FrozenGoalTexts);
    _episode = new EpisodeCore(EpisodeCore.FrozenEpisodeIds, EpisodeCore.FrozenEpisodeNames, EpisodeCore.FrozenE
    _field.StartRun(0);
    _goal.StartRun();
    Text caption = null;
    Transform homestead = _pages.ContainsKey("S01_Homestead") ? _pages["S01_Homestead"].transform : null;
    if (homestead != null)
    {
        Transform label = homestead.Find("TeachCaptionLabel");
        if (label != null) caption = label.GetComponent&lt;Text&gt;();
    }
    double last = 0.0;
    for (int i = 0; i &lt; PagesGate.TeachBeats.Length; i++)
    {
        TeachBeat beat = PagesGate.TeachBeats[i];
        double wait = beat.T - last;
        if (wait &gt; 0) yield return new WaitForSeconds((float)wait);
        last = beat.T;
        RunTeachBeat(_field, _goal, _episode, i);
        if (caption != null) caption.text = Locale.ByKey(beat.CaptionKey);
    }
    CaptureAndSave();
}
/// &lt;summary&gt;One beat of the frozen six-beat opening on the real cores (the
/// same sequence PagesGate.RunTeachDrive replays in logical time).&lt;/summary&gt;
public static void RunTeachBeat(FieldCore field, GoalCore goal, EpisodeCore episode, int beatIndex)
{
```

```
            switch (beatIndex)
            {
                case 0: // 0.15 hook caption
                    field.SimTime = PageFrozen.CaptionT;
                    field.Log("caption", PagesGate.TeachBeats[0].CaptionMirror);
                    break;
                case 1: // 1.00 first till
                    field.SimTime = FieldCore.FirstTillT;
                    field.TillPlot(0);
                    break;
                case 2: // 1.50 first feedback triple
                    field.SimTime = FieldCore.FirstPlantT;
                    field.PlantPlot(0, 0);
                    break;
                case 3: // 2.00 till #2, then 2.50 till #3 + first harvest + goal double-fire
                    field.SimTime = PageFrozen.TeachSecondTillT;
                    field.TillPlot(1);
                    field.SimTime = GoalCore.FirstGoalBeatT;
                    field.TillPlot(2);
                    field.GrowTo(GoalCore.FirstGoalBeatT);
                    field.HarvestPlot(0);
                    goal.SimTime = GoalCore.FirstGoalBeatT;
                    goal.BeatGoals(field, episode);
                    break;
                case 4: // 5.50 episode 1 unlock (pilot free)
                    episode.SimTime = EpisodeCore.EpisodeUnlockT;
                    episode.UnlockEpisode(1);
                    goal.SimTime = EpisodeCore.EpisodeUnlockT;
                    goal.BeatGoals(field, episode);
                    break;
                case 5: // 6.00 episode 1 starts, three rows in; 7.00 they ripen (+300 shi);
                        // 8.00 E1 clears the 300 target - win + cliffhanger + one double offer
                    episode.SimTime = 6.00;
                    episode.StartEpisode(1);
                    field.SimTime = 6.00;
                    field.PlantPlot(0, 0);
                    field.PlantPlot(1, 1);
                    field.PlantPlot(2, 2);
                    field.GrowTo(7.00);
                    long g0 = field.HarvestPlot(0);
                    long g1 = field.HarvestPlot(1);
                    long g2 = field.HarvestPlot(2);
                    episode.SimTime = 7.00;
                    episode.CreditGrain(g0);
                    episode.CreditGrain(g1);
                    episode.CreditGrain(g2);
                    goal.SimTime = 7.00;
                    goal.BeatGoals(field, episode);
                    episode.SimTime = EpisodeCore.CliffhangerT;
                    episode.ResolveEpisode(1, episode.EpisodeTargets[0]);
```

```
                    goal.SimTime = EpisodeCore.CliffhangerT;
                    goal.BeatGoals(field, episode);
                    break;
                }
            }
            /// &lt;summary&gt;Save wiring: mirror the field grain ledger + the completed
            /// episodes into the SaveFace, then land it through ISaveService. The T009
            /// contract wired here: the AdCore ExportLedger IS the save.adcaps face (the
            /// c1 host carries no live ad ledger - the section lands empty until the
            /// presenter wires the ad slots in a later slice).&lt;/summary&gt;
            public void CaptureAndSave()
            {
                PagesGate.SaveFace face = new PagesGate.SaveFace();
                if (_field != null) face.Grain = _field.GrainTotal;
                if (_episode != null)
                    for (int e = 1; e &lt;= EpisodeCore.EpisodeCount; e++)
                        if (_episode.EpisodeCompleted[e - 1]) face.EpisodesCompleted.Add(e);
                ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, PagesGate.SaveCodec.Compose(face));
            }
        }
        /// &lt;summary&gt;Full-stretch helper for backdrop rects (anchor both corners, zero offsets).&lt;/summary&gt;
        public static class RectTransformExtensions
        {
            public static RectTransform StretchFull(this RectTransform rt)
            {
                rt.anchorMin = Vector2.zero;
                rt.anchorMax = Vector2.one;
                rt.offsetMin = Vector2.zero;
                rt.offsetMax = Vector2.zero;
                return rt;
            }
        }
    }
    // ShareCodecCore.cs
    // ShareCodecCore.cs - S1 T008 share-codec core for ExileFarm (B1276 kit-port).
    // Ported under the kit-port law: every frozen value stays verbatim - zero new tuning.
    // Frozen anchors (all traceable to the E4 kit, Design/evidence/m6_exilefarm_graybox_kit/):
    //   - code prefix "EF1-" = kit ShareCard face verbatim (the share-code family mark)
    //   - compose face = kit ShareCard verbatim: one uppercase hex digit per owned
    //     land-dex terrain (i.ToString("X") over the twelve-terrain roster) - the
    //     episode-achievement land-dex card, never a score form (B904 guardrail #2
    //     analog; requirement M08 "EF1- prefix share code, X032 offline shape (zero
    //     server), compose/decode round-trip", S06 ShareCodec ComposeCard/DecodeCard)
    //   - share-form ban = kit ShareFormTokens verbatim: score / points / result / rank
    //     never appear in any share payload face
    //   - forbidden IP tokens = kit ForbiddenTokens verbatim (U033 / B1227 admission:
    //     the trope is used, the IP is not)
    //   - card gate = kit GrayboxBot face verbatim: a won run fires the card only when
    //     the land dex wall carries at least 8 of the 12 terrains
    //   - share_card event detail = kit verbatim: "episode achievement card
```

```
//     code=&lt;EF1-...&gt; land_dex=N/12 (show off your estate)"
// Structural S1 extension vs the kit: the decode face (M08 requires a true
// compose/decode round-trip; the kit only composed). The X032 offline shape is
// structural: the code alone restores the owned-terrain set with zero server and
// zero external state (a fresh codec instance decodes any card identically).
// Decode is strict: wrong prefix / empty payload / non-hex char / out-of-roster
// digit / duplicate digit all reject with exactly one event each. Zero threshold
// change. State consumption: the codec observes the CodexCore land dex; it never
// mutates anything. Pure C#: zero UnityEngine.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.ExileFarm
{
    public sealed class ShareCodecEvent
    {
        public double t;
        public string type;
        public string detail;
        public ShareCodecEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detai
    }
    public sealed class ShareCodecCore
    {
        // ---- frozen anchors (kit mirrors) ----
        public const string CodePrefix = "EF1-";      // kit ShareCard prefix verbatim
        public const int FrozenTerrainCount = 12;     // twelve-terrain roster mirror
        public const int CardGateTerrainWall = 8;     // kit bot card gate verbatim: &gt;= 8 of 12
        public static readonly string[] ShareFormTokens = { "score", "points", "result", "rank" };
        public static readonly string[] ForbiddenTokens =
        {
            "guilty_wife", "criminal_wife", "fa_pei_bian_guan", "zhan_shen_zhang_fu",
            "beheaded", "execution_ground", "torture", "prison_chain"
        };
        public double SimTime;
        public readonly List&lt;ShareCodecEvent&gt; Events = new List&lt;ShareCodecEvent&gt;();
        public int CardsComposed, CardsDecoded, CardsDeclined, CardsRejected;
        public ShareCodecCore(string[] terrainRoster)
        {
            if (terrainRoster == null)
                throw new ArgumentException("terrain roster must carry exactly " + FrozenTerrainCount + " rows");
            if (terrainRoster.Length != FrozenTerrainCount)
                throw new ArgumentException("terrain roster must carry exactly " + FrozenTerrainCount
                    + " rows, got " + terrainRoster.Length);
        }
        public void Log(string type, string detail) { Events.Add(new ShareCodecEvent(SimTime, type, detail)); }
        public double TimeOf(string type)
        {
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
            return -1.0;
```

```
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
// ==== dual-token scan faces (kit ContainsAny over the frozen mirrors) ====
public static bool ContainsShareFormToken(string text)
{
    if (text == null) return false;
    for (int i = 0; i &lt; ShareFormTokens.Length; i++) if (text.Contains(ShareFormTokens[i])) return true;
    return false;
}
public static bool ContainsForbiddenToken(string text)
{
    if (text == null) return false;
    for (int i = 0; i &lt; ForbiddenTokens.Length; i++) if (text.Contains(ForbiddenTokens[i])) return true;
    return false;
}
// ==== compose from an owned set: the pure restore face (no gate, no
// events) - the X032 offline half and the round-trip pair of DecodeCard.
// Input is the sorted owned array (CodexCore.QueryTerrainAll shape).
public static string ComposeOwned(int[] ownedTerrain)
{
    if (ownedTerrain == null) return null;
    StringBuilder sb = new StringBuilder(CodePrefix);
    for (int i = 0; i &lt; ownedTerrain.Length; i++)
        sb.Append(ownedTerrain[i].ToString("X", CultureInfo.InvariantCulture));
    return sb.ToString();
}
// ==== compose the run card = kit ShareCard face verbatim behind the kit
// bot gate: a won run shows the estate off only when the land dex wall is
// populated enough. The card is an episode-achievement land-dex card; the
// share-form words (score/points/result/rank) stay out of every face.
public string ComposeCard(CodexCore codex)
{
    if (codex == null)
    {
        CardsRejected++;
        Log("share_rejected", "no dex to show off");
        return null;
    }
    int[] owned = codex.QueryTerrainAll();
```

```
            if (owned.Length &lt; CardGateTerrainWall)
            {
                CardsDeclined++;
                Log("share_declined", "land dex " + owned.Length + "/" + FrozenTerrainCount
                    + " - the estate is not ready to show off yet");
                return null;
            }
            string code = ComposeOwned(owned);
            CardsComposed++;
            Log("share_card", "episode achievement card code=" + code + " land_dex=" + owned.Length
                + "/" + FrozenTerrainCount + " (show off your estate)");
            return code;
        }
        // ==== decode = structural extension (M08 round-trip face): strict parse,
        // zero server, zero external state. Accepts any digit order, rejects wrong
        // prefix / empty payload / non-uppercase-hex / out-of-roster digits /
        // duplicate digits - one share_decode_rejected event per reject.
        public int[] DecodeCard(string code)
        {
            if (code == null) return RejectDecode("null code");
            if (!code.StartsWith(CodePrefix, StringComparison.Ordinal))
                return RejectDecode("wrong or missing EF1- prefix: '" + code + "'");
            string payload = code.Substring(CodePrefix.Length);
            if (payload.Length == 0) return RejectDecode("empty payload after the EF1- prefix");
            var set = new HashSet&lt;int&gt;();
            for (int i = 0; i &lt; payload.Length; i++)
            {
                char ch = payload[i];
                int v;
                if (ch &gt;= '0' &amp;&amp; ch &lt;= '9') v = ch - '0';
                else if (ch &gt;= 'A' &amp;&amp; ch &lt;= 'F') v = ch - 'A' + 10;
                else return RejectDecode("non-hex char '" + ch + "' at payload slot " + i);
                if (v &gt;= FrozenTerrainCount)
                    return RejectDecode("digit '" + ch + "' is outside the twelve-terrain roster");
                if (!set.Add(v)) return RejectDecode("duplicate terrain digit '" + ch + "'");
            }
            var list = new List&lt;int&gt;(set);
            list.Sort();
            CardsDecoded++;
            Log("share_decode", "card decoded: " + list.Count + " terrains restored (offline shape, zero server)");
            return list.ToArray();
        }
        int[] RejectDecode(string reason)
        {
            CardsRejected++;
            Log("share_decode_rejected", reason);
            return null;
        }
    }
}
```